Colección de Ejercicios Prácticos de Auditoría de Seguridad Informática (Nivel 3)

Nota previa: Todos estos ejercicios deben realizarse en un entorno de laboratorio **seguro y aislado** (red interna o *host-only*), con máquinas virtuales si es posible. No se debe **atacar redes o sistemas fuera del laboratorio**. Antes de cada práctica, tome *snapshots* de los sistemas por seguridad, y documente todas las evidencias (comandos utilizados, salidas obtenidas, capturas de pantalla).

A continuación se presentan *20 ejercicios prácticos* utilizando herramientas sencillas de auditoría de seguridad informática, ordenados aproximadamente por nivel de dificultad (de básico a más avanzado). Cada ejercicio incluye el objetivo, instrucciones paso a paso, comandos necesarios, riesgos asociados con medidas de mitigación, resultados esperados y una solución guiada que explica qué debería observar el alumno.

Ejercicio 1: Reconocimiento básico de red (Ping y Traceroute)

Herramientas: Ping, Traceroute/Tracert
Objetivo: Utilizar herramientas básicas del sistema operativo (ping y traceroute) para comprobar la conectividad de hosts en la red local y trazar la ruta que siguen los paquetes hasta un destino. Esto permite mapear de forma inicial la red y detectar la presencia de los equipos.

Instrucciones:

1. **Comprobar conectividad con ping:** Identifique la dirección IP de otro host en la red de laboratorio (por ejemplo, la IP de la puerta de enlace o de otra máquina virtual). Abra una terminal y ejecute ping <IP_destino>. Observe la respuesta: el comando enviará mensajes ICMP Echo Request al host destino y mostrará si hay **respuesta (Echo Reply)** indicando que el host está alcanzable.

2. Pruebe a hacer ping a una IP *inexistente* o a un host apagado para ver la diferencia (debería obtener *tiempo de espera agotado* o *host inaccesible*). Esto le ayuda a distinguir cuándo un equipo está **activo** en la red.

3. **Traza de ruta con traceroute/tracert:** Ahora ejecute la traza de ruta para el mismo destino. En Linux use traceroute <IP_destino> y en Windows use tracert <IP_destino>. Este comando enumerará los **saltos (hops)** intermedios que recorren los paquetes para llegar al destino.

4. Revise la lista de hops que devuelve traceroute: normalmente comenzará con la IP de su **puerta de enlace** local (router) y listará cada router intermedio hasta el destino. En una red local pequeña puede que solo aparezca el router y luego directamente el host destino (o incluso solo el destino si está en la misma subred).

5. Anote los resultados: cuántos saltos hubo, cuál es el tiempo de respuesta promedio del ping, etc. Si su laboratorio tiene varias subredes o un acceso a Internet controlado, traceroute le mostrará por dónde salen los paquetes de su red.

Comandos necesarios:

ping 192.168.1.100 # Envía paquetes ICMP Echo Request a 192.168.1.100

traceroute 192.168.1.100 # (En Linux) Muestra la ruta de saltos hasta 192.168.1.100

tracert 192.168.1.100 # (En Windows) Comando equivalente de traceroute

Riesgos asociados y mitigación:

- **Riesgo:** El uso de ping y traceroute es inofensivo en términos de carga, pero enviar pings continuos a muchos hosts podría considerarse **reconocimiento no autorizado** fuera del laboratorio. En redes reales, estos comandos pueden revelar información de topología a un atacante.
  Mitigación: Utilizarlos **solo en entornos controlados o con permiso**. En producción, se pueden **filtrar ICMP** en el cortafuegos para que externos no mapeen la red fácilmente, aunque internamente son útiles para diagnóstico.

Resultados esperados:

- Con ping, debería recibir respuestas tipo 64 bytes from <IP_destino>: icmp_seq=1 ttl=... time=... ms en Linux (o *Respuesta desde <IP>: bytes=32 tiempo=<X>ms TTL=<Y>* en Windows). Esto indica que el host está **activo** y devuelve paquetes en X milisegundos. Si el host no responde, verá mensajes de tiempo de espera agotado (*Request timed out*).

- Con traceroute/tracert, debería ver enumeradas las IP de los routers intermedios. Por ejemplo, *1 <IP_router_local> ...*, *2 <IP_router_proveedor> ...*, etc., hasta eventualmente la IP destino. En una red local simple, es posible que solo aparezca un salto (el router) y luego el destino. Cada línea muestra también el tiempo que tardaron los paquetes a cada hop.

Solución guiada:
Al ejecutar ping a otro equipo del laboratorio, el alumno observará que obtiene respuestas con tiempos muy bajos (p. ej. 1-2 ms en una LAN), confirmando conectividad. Esto verifica que la red entre su máquina y la de destino funciona correctamente. Si el ping no responde, significa que el host está caído, no accesible, o que podría estar bloqueando ICMP.

Con traceroute, el alumno verá, por ejemplo, primero la dirección de su **puerta de enlace** (demostrando que ese es el primer salto en la ruta) seguida quizás de direcciones de otros routers. Cada salto añade algo de latencia. Si el destino está en la misma subred, es posible que traceroute muestre solo un salto o directamente el destino. En cualquier caso, este comando proporciona una **visión de la ruta** que siguen los datos.

En una auditoría, ping y traceroute se usan para **descubrir hosts activos y la estructura de la red**. Son herramientas sencillas pero importantes: un auditor puede detectar qué equipos responden (y por tanto están en operación) y con traceroute puede inferir la presencia de múltiples segmentos o rutas. Por ejemplo, si traceroute muestra varios saltos internos, sugiere una red con varias capas. Esto ayuda a planificar fases posteriores de la auditoría.

El alumno debe concluir que estas herramientas confirman **qué hosts están en la red** y **cómo están conectados**, sirviendo de punto de partida para análisis más profundos. Dado que no ofrecen cifrado ni autenticación, también verá que son susceptibles de ser detectadas o bloqueadas (algunos sistemas podrían no responder a pings deliberadamente). En entornos seguros, es común que el cortafuegos bloquee ICMP desde fuera, por lo que un auditor externo podría no mapear fácilmente la red con ping; sin embargo, dentro de la red de laboratorio sí obtiene las respuestas esperadas.

ping 192.168.0.7

```
~/Documents/IFCT0109 ❯ ping 192.168.0.7
PING 192.168.0.7 (192.168.0.7) 56(84) bytes of data.
64 bytes from 192.168.0.7: icmp_seq=1 ttl=64 time=1.65 ms
64 bytes from 192.168.0.7: icmp_seq=2 ttl=64 time=2.08 ms
64 bytes from 192.168.0.7: icmp_seq=3 ttl=64 time=1.89 ms
^C
--- 192.168.0.7 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.647/1.870/2.080/0.177 ms
```

traceroute 192.168.0.7

```
~/Documents/IFCT0109 ❯ traceroute 192.168.0.7
traceroute to 192.168.0.7 (192.168.0.7), 30 hops max, 60 byte packets
 1  192.168.0.7 (192.168.0.7)  2.153 ms  2.329 ms  2.576 ms
```

tracert 192.168.0.7

```
PowerShell 7.5.4
PS C:\Users\2-DAW> tracert 192.168.0.7

Traza a 192.168.0.7 sobre caminos de 30 saltos como máximo.

  1     1 ms     1 ms     1 ms  192.168.0.7

Traza completa.
```

sudo tcpdump -i eth0 icmp

```
~/Documents/IFCT0109 ❯ sudo tcpdump -i eth0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:49:48.840741 IP 192.168.0.7 > 192.168.0.10: ICMP echo request, id 257, seq 10, length 64
09:49:48.840780 IP 192.168.0.10 > 192.168.0.7: ICMP echo reply, id 257, seq 10, length 64
09:49:49.841855 IP 192.168.0.7 > 192.168.0.10: ICMP echo request, id 257, seq 11, length 64
09:49:49.841896 IP 192.168.0.10 > 192.168.0.7: ICMP echo reply, id 257, seq 11, length 64
09:49:50.843302 IP 192.168.0.7 > 192.168.0.10: ICMP echo request, id 257, seq 12, length 64
09:49:50.843330 IP 192.168.0.10 > 192.168.0.7: ICMP echo reply, id 257, seq 12, length 64
09:49:51.845630 IP 192.168.0.7 > 192.168.0.10: ICMP echo request, id 257, seq 13, length 64
09:49:51.845655 IP 192.168.0.10 > 192.168.0.7: ICMP echo reply, id 257, seq 13, length 64
```

Ejercicio 2: Enumeración de equipos Windows/SMB con NBTScan

Herramienta: NBTScan
Objetivo: Utilizar **NBTScan** para descubrir equipos Windows o con servicios SMB en la red local, enumerando sus nombres **NetBIOS** y otros datos de compartición. Esto permite obtener información de nombres de host y dominios de Windows de forma rápida. (NetBIOS es un servicio de compartición de archivos e impresoras que publica nombres de equipo en redes locales).

Instrucciones:

1. Elija un rango de IP dentro de la red de laboratorio que quiera escanear. Por ejemplo, si su red es 192.168.1.0/24, ese será el rango objetivo.
2. Abra una terminal en la máquina atacante (Linux con NBTScan instalado, o Windows con la versión correspondiente). Ejecute el comando de escaneo especificando el rango completo. Por ejemplo: nbtscan 192.168.1.0/24. Esto enviará consultas NetBIOS Name Service a cada dirección IP en ese rango.
3. Espere a que finalice el escaneo. NBTScan listará cada IP que respondió junto con información como: **Nombre NetBIOS** (nombre del equipo), el grupo de trabajo o dominio al que pertenece, y la dirección MAC (si está en la misma subred).
4. Revise la salida: anote los nombres de host descubiertos. Por ejemplo, puede ver algo como 192.168.1.5 MACHINE1 <00> UNIQUE Workstation Service o similar. Esto indicaría que en 192.168.1.5 hay un host cuyo nombre NetBIOS es MACHINE1.
5. Si la red es grande, puede filtrar resultados. Pero en un /24 la salida será manejable. Verifique cuántos hosts respondieron. Si sabe de algún equipo Windows específico, busque su IP en la lista y corrobore que el nombre coincide con lo esperado.
6. (Opcional) Ejecute nbtscan -v 192.168.1.0/24 con la opción *verbose* para obtener información adicional de cada respuesta, como el listado completo de nombres NetBIOS registrados (puede incluir entradas del servidor de archivos, impresoras, etc.).

Comandos necesarios:

nbtscan 192.168.1.0/24 # Escanear todo el rango 192.168.1.* en busca de nombres NetBIOS

nbtscan -v 192.168.1.0/24 # Modo verbose: muestra detalles adicionales de NetBIOS

Riesgos asociados y mitigación:

- **Riesgo:** El escaneo NetBIOS es básicamente un **sondeo de información**; no daña, pero **revela nombres de máquinas y dominios** a quien escanee. Un atacante podría usar esto para perfilar la red (saber nombres de PCs, roles, etc.).
  Mitigación: Deshabilitar el servicio **NetBIOS/SMB** en estaciones donde no sea necesario, o configurar los cortafuegos locales para no responder a estas peticiones desde orígenes no confiables. En entornos modernos, el protocolo LLMNR/NetBIOS puede sustituirse por DNS interno más seguro.

Resultados esperados:

- La herramienta listará las IP activas que tengan compartición Windows habilitada. Por ejemplo:
- 192.168.1.5 WORKSTATION1 <00> UNIQUE Workstation Service
- 192.168.1.5 MYGROUP <00> GROUP Domain/Workgroup Name
- 192.168.1.5 WORKSTATION1 <20> UNIQUE File Server Service
- 192.168.1.10 SERVER-DB <00> UNIQUE Workstation Service
- 192.168.1.10 MYGROUP <00> GROUP Domain/Workgroup Name
- 192.168.1.10 SERVER-DB <20> UNIQUE File Server Service
- ...

En este ejemplo hipotético, NBTScan encontró dos hosts: uno con IP .5 llamado WORKSTATION1 y otro con IP .10 llamado SERVER-DB. Ambos están en el grupo de trabajo/dominio MYGROUP. La marca <20> indica que esos hosts tienen el servicio de servidor de ficheros habilitado (SMB activo).

- Es esperable obtener nombres de equipo legibles, a veces incluyendo roles (por ejemplo *SERVER-DB* sugiere un servidor de base de datos). Si ningún host tuviera NetBIOS activo, NBTScan terminaría sin resultados significativos. En una red Windows típica, al menos los controladores de dominio o PCs cliente responden.

Solución guiada:
El alumno deberá haber identificado correctamente varios equipos Windows activos en la red a partir de la salida de NBTScan. Cada entrada proporciona el **nombre NetBIOS** del equipo, que a menudo coincide con el nombre de la máquina configurado por la empresa

(dando pistas, como *SERVER-DB* en el ejemplo sugiere un servidor de base de datos). También aparece el **grupo de trabajo o dominio** (por ejemplo, *MYGROUP* podría ser un dominio Windows). Esta información es valiosa en una auditoría: permite saber qué equipos existen y cómo se llaman, lo que puede indicar su función o departamento.

Por ejemplo, si en los resultados aparece un nombre como *BDC-FileServer*, uno podría deducir que es un servidor de ficheros posiblemente importante. Conocer el dominio (si en vez de MYGROUP fuera, digamos, *EMPRESA*), confirmaría que hay un dominio Windows llamado EMPRESA al que esos equipos pertenecen.

En términos de seguridad, el ejercicio evidencia que **NetBIOS/SMB expone metadatos de la red** fácilmente. Un auditor o atacante no autenticado pudo listar nombres de host y con ello perfilar la red. La **medida de mitigación** comentada (deshabilitar NetBIOS si no es necesario) se basa en que en redes modernas, este protocolo es legado y, si no se requiere compatibilidad con sistemas antiguos, mejor apagarlo para no dar esta información de más. También limitar la respuesta a estas peticiones en el firewall de host o red hará que un escaneo de este tipo desde fuera no devuelva nada.

El alumno habrá aprendido que con una simple herramienta puede **descubrir la presencia de máquinas Windows y sus nombres** en cuestión de segundos, algo que luego puede aprovechar para otros ejercicios (por ejemplo, identificar contra qué equipos lanzar un escaneo de puertos más detallado, o qué nombre de host buscar en credenciales). Es una técnica típica en la fase de reconocimiento de una auditoría de seguridad.

nbtscan 192.168.0.0/24

```
~/Documents/IFCT0109 ❯ nbtscan 192.168.0.0/24
Doing NBT name scan for addresses from 192.168.0.0/24

IP address       NetBIOS Name     Server    User             MAC address
-------------------------------------------------------------------------------
192.168.0.5      RNP00267399578C  <server>  <unknown>        00:26:73:99:57:8c
192.168.0.28     <unknown>                  <unknown>        78:ac:c0:c2:c6:66
192.168.0.31     DESKTOP-M7CM9BE  <server>  <unknown>        dc:4a:3e:7c:25:3e
192.168.0.54     DESKTOP-AFJF0BS  <server>  <unknown>        24:be:05:21:18:3b
192.168.0.91     <unknown>                  <unknown>        3c:d9:2b:74:88:8e
192.168.0.89     PA1-LINKSYS                <unknown>        c0:56:27:eb:b6:7f
192.168.0.115    <unknown>                  <unknown>        ac:16:2d:0f:c1:c7
192.168.0.117    PC-A02           <server>  <unknown>        98:ee:cb:46:94:f7
192.168.0.110    PC-SERGIO        <server>  <unknown>        2c:41:38:ac:7d:1c
192.168.0.138    <unknown>                  <unknown>        24:be:05:04:91:30
192.168.0.70     PC-A06           <server>  <unknown>        dc:4a:3e:8d:c1:b5
```

Ejercicio 3: Escaneo de puertos y servicios con Nmap

Herramienta: Nmap
Objetivo: Emplear **Nmap** para explorar la red y **descubrir puertos abiertos** en los hosts activos, identificando qué servicios y versiones están corriendo. Nmap es una herramienta ampliamente utilizada de exploración de red y **auditoría de seguridad** que permite mapear rápidamente puertos y servicios. El objetivo es aprender a usar sus funciones básicas para obtener un panorama de las potenciales entradas en el sistema auditado.

Instrucciones:

1. **Detección de hosts activos:** Primero, use Nmap para detectar qué hosts del rango están encendidos. Por ejemplo, para la subred 192.168.1.0/24, ejecute un **ping sweep** con Nmap: nmap -sn 192.168.1.0/24. Esto hará que Nmap envíe pings/ARP a cada dirección y liste las que respondan. Anote cuáles IP están activas (Nmap las mostrará como "Host is up").

2. **Escaneo de puertos en un host:** Elija uno de los hosts activos (por ejemplo, el que encontró en el Ejercicio 2 con un nombre interesante o la IP de un servidor conocido). Ejecute un escaneo de puertos **TCP** en ese host. Un comando común es nmap -sS -p1-1000 -sV <IP_objetivo>. Aquí, -sS realiza un escaneo TCP SYN (semi-intrusivo pero rápido), -p1-1000 limita el escaneo a los primeros 1000 puertos (los más comunes) y -sV intenta **detectar la versión del servicio** en cada puerto abierto. Puede omitir el rango -p para que Nmap use su puerto por defecto (los 1000 más comunes).

3. **Esperar y analizar la salida:** Nmap listará cada puerto probado y su estado: *open* (abierto), *closed* (cerrado) o *filtered* (filtrado por firewall). Para puertos abiertos, mostrará el nombre del servicio (por ejemplo, 80/tcp http) y, con -sV, detalles de versión (por ejemplo "Apache httpd 2.4.41"). Revise los resultados.

4. **Escaneo de varios hosts:** Puede repetir el escaneo de puertos para otros IP activos. Alternativamente, Nmap permite escanear varios hosts a la vez (por ejemplo nmap -sS -p 1-1000 -sV 192.168.1.5 192.168.1.10). Pero tenga cuidado con el tiempo; escanear muchos puertos en muchos hosts puede tardar. Para el laboratorio, enfoque en 1 o 2 máquinas clave.

5. **Escaneo de UDP (opcional):** Si desea, pruebe un escaneo de puertos UDP en el objetivo (usando -sU). Esto detecta servicios UDP (como DNS en 53/udp). Tenga en cuenta que UDP es más lento de escanear y a veces menos confiable en resultados. Puede intentar nmap -sU -p 53,161 <IP_objetivo> para ver si DNS (53) o SNMP (161) están abiertos, a modo de ejemplo.

6. **Salida en formato legible:** Asegúrese de entender la salida: por cada puerto abierto, verá algo como PORT STATE SERVICE VERSION. Por ejemplo: 22/tcp open ssh OpenSSH 8.2p1 Debian. Esto le indica puerto, estado, tipo de servicio y versión detectada.

7. Documente los hallazgos: puertos abiertos y servicios. Piense qué implicaciones tiene cada servicio (ej.: si ve 22/tcp ssh abierto, significa que podría intentar login SSH en otro ejercicio; si ve 80/tcp http, hay un servidor web que podría ser auditado más adelante, etc.).

Comandos necesarios:

nmap -sn 192.168.1.0/24 # Descubrir hosts activos en la subred (ping scan)

nmap -sS -p1-1000 -sV 192.168.1.5 # Escaneo SYN de puertos TCP 1-1000 con detección de versión en host .5

nmap -A 192.168.1.5 # Escaneo completo (agresivo: puertos + versiones + OS) host .5

nmap -sU -p53,161 192.168.1.5 # Escanear solo puertos UDP 53 y 161 en host .5

Nota: La opción -A activa varias funciones juntas (detectar SO, versiones y scripts básicos), puede usarse para simplificar, aunque es más ruidosa y lenta.

Riesgos asociados y mitigación:

- **Riesgo:** El escaneo de puertos es **activo**; puede generar mucho tráfico y ser detectado por sistemas de defensa (IDS/IPS) como una posible intrusión. Además, un escaneo muy agresivo o mal configurado podría sobrecargar servicios o provocar **logs voluminosos** en los equipos destino.
  Mitigación: Realizar los escaneos en ventanas programadas (por ejemplo, horas de bajo uso) y limitar el alcance: escanear solo lo necesario y a velocidad moderada (Nmap permite ajustar el *timing*). Siempre obtener **permiso explícito** para escanear sistemas de terceros. En entornos seguros, configurar firewalls para filtrar o limitar peticiones de escaneo (p.ej., bloquear puertos no autorizados) puede mitigar la exploración maliciosa.

Resultados esperados:

- Para el ping sweep inicial (-sn), el resultado típico es una lista de IP activas. Por ejemplo:
- Nmap scan report for 192.168.1.5
- Host is up (0.0010s latency).
- Nmap scan report for 192.168.1.10
- Host is up (0.0008s latency).
- Nmap scan report for 192.168.1.12
- Host is up (0.0021s latency).

- …

Esto indica que las IP .5, .10, .12 respondieron (están *up*).

- Para el escaneo de puertos en, digamos, 192.168.1.5, podría obtener una salida como:
- PORT STATE SERVICE VERSION
- 22/tcp open ssh OpenSSH 8.2p1 Ubuntu
- 80/tcp open http Apache httpd 2.4.41
- 139/tcp open netbios-ssn Samba smbd 4.11.6
- 445/tcp open microsoft-ds Windows Server 2019 microsoft-ds
- 3306/tcp open mysql MySQL 5.7.31
- 8080/tcp open http-proxy Apache Tomcat/Coyote JSP engine 1.1
- MAC Address: 00:0C:29:4D:8E:35 (VMware)

(Esto es un ejemplo hipotético de un servidor con SSH, web, compartición Windows, base de datos MySQL y un Tomcat). Nmap identificó varios servicios y hasta adivinó que es un VMware por la MAC.

- El alumno debe esperar ver **al menos uno o dos puertos abiertos** en cada host típico. Por ejemplo, en una PC Windows cliente podría encontrar 139/445 (compartición) y tal vez 3389 (Escritorio Remoto) si está activo. En un Linux, típicamente 22 (SSH) y quizás algún 80 si tiene un servidor web.
- Si se prueba -A o -O, Nmap intentará también adivinar el **sistema operativo** por patrones de red. Podría decir *OS: Linux 5.x* o *Windows 10*, etc., con cierto nivel de fiabilidad.

Solución guiada:
Tras realizar el escaneo, el alumno habrá reunido una lista de **puertos abiertos** y **servicios** en los sistemas de la red. Esto es fundamental en una auditoría, pues cada puerto abierto representa una superficie de ataque potencial. Por ejemplo, si Nmap reveló un puerto 3306/tcp abierto con MySQL, el auditor sabe que ese servidor de base de datos podría ser investigado (¿está con contraseña débil? ¿permite conexiones remotas?). Si vio 139/tcp y 445/tcp abiertos, implica que la máquina tiene compartición de archivos SMB activa; combinando con Ejercicio 2, sabría el nombre NetBIOS de esa máquina y podría intentar conectar para enumerar recursos compartidos.

Un resultado interesante sería descubrir servicios inesperados. Por ejemplo, encontrar un puerto 8080 con Tomcat (como en el ejemplo) puede indicar una **aplicación web de administración** que quizás esté desprotegida. El alumno debe preguntarse por cada hallazgo: *¿Este servicio debería estar abierto?* y *¿qué vulnerabilidades conocidas podría tener?*.

Justamente, Nmap ayudó a **identificar versiones** (como Apache 2.4.41, MySQL 5.7.31). Con esa información, uno puede luego consultar bases de datos de CVEs para ver si esas versiones son vulnerables.

En cuanto a la técnica, el alumno habrá experimentado varios modos de Nmap. Verá que un escaneo SYN (-sS) es rápido y discreto comparado con un Connect full (-sT). También notará que para detectar versiones se envían paquetes adicionales; por eso Nmap pudo listar *OpenSSH 8.2p1* en lugar de solo "ssh". Internamente, hace pruebas de banner y comportamiento.

Es importante destacar que, en entornos reales, un **IDS/IPS** podría registrar estos escaneos (por la sucesión rápida de conexiones). Sin embargo, en el laboratorio no hay tal restricción, así que seguramente todos los puertos abiertos fueron detectados. La mitigación mencionada se refiere a prácticas como limitar escaneos y escanear con permiso, ya que lanzar Nmap en una red corporativa sin aviso podría disparar alarmas. **En auditorías autorizadas**, normalmente se coordina para que el SOC ignore ciertos escaneos durante la ventana de test.

Finalmente, el alumno debe comprender que ha creado un *mapa de servicios*: por ejemplo, "El Host A (192.168.1.5, Linux) tiene SSH, Apache y MySQL; el Host B (192.168.1.10, Windows) tiene compartición SMB y escritorio remoto". Este mapa será la base para los siguientes ejercicios (por ejemplo, probar credenciales por fuerza bruta en SSH o SMB, analizar la aplicación web en Apache, etc.). En resumen, Nmap les dio un **inventario de puertas** por las cuales podrían intentar entrar, y evidenció la importancia de **minimizar puertos abiertos** o protegerlos adecuadamente en los sistemas auditados.

nmap -sn 192.168.0.0/24

```
~/Documents/IFCT0109 ❯ nmap -sn 192.168.0.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 09:57 CET
Nmap scan report for 192.168.0.1
Host is up (0.0013s latency).
MAC Address: 00:A0:26:D2:68:9A (Teldat)
Nmap scan report for 192.168.0.5
Host is up (0.00098s latency).
MAC Address: 00:26:73:99:57:8C (Ricoh Company)
Nmap scan report for 192.168.0.7
Host is up (0.00087s latency).
MAC Address: 08:00:27:44:AC:A5 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.0.22
Host is up (0.0027s latency).
MAC Address: 08:00:27:CB:65:FF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.0.25
Host is up (0.0018s latency).
MAC Address: EC:71:DB:A7:4B:40 (Reolink Innovation Limited)
Nmap scan report for 192.168.0.28
Host is up (0.0017s latency).
MAC Address: 78:AC:C0:C2:C6:66 (Hewlett Packard)
Nmap scan report for 192.168.0.31
Host is up (0.0019s latency).
MAC Address: DC:4A:3E:7C:25:3E (Hewlett Packard)
```

nmap -sS -p1-1000 -sV 192.168.0.7

```
~/Documents/IFCT0109 ❯ nmap -sS -p1-1000 -sV 192.168.0.7                                38s
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 09:59 CET
Nmap scan report for 192.168.0.7
Host is up (0.0013s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.65 ((Debian))
MAC Address: 08:00:27:44:AC:A5 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.68 seconds
```

nmap -A 192.168.0.7

```
~/Documents/IFCT0109 ❯ nmap -A 192.168.0.7
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 10:14 CET
Nmap scan report for 192.168.0.7
Host is up (0.0018s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.65 ((Debian))
|_http-server-header: Apache/2.4.65 (Debian)
|_http-title: Apache2 Debian Default Page: It works
| vulners:
|   cpe:/a:apache:http_server:2.4.65:
|       CNVD-2024-36391 9.8     https://vulners.com/cnvd/CNVD-2024-36391
|       CNVD-2024-36388 9.8     https://vulners.com/cnvd/CNVD-2024-36388
|       CNVD-2022-41640 9.8     https://vulners.com/cnvd/CNVD-2022-41640
|       CNVD-2020-46280 9.8     https://vulners.com/cnvd/CNVD-2020-46280
|       1337DAY-ID-34882        9.8     https://vulners.com/zdt/1337DAY-ID-3
|       FD2EE3A5-BAEA-5845-BA35-E6889992214F    9.1     https://vulners.com/
845-BA35-E6889992214F   *EXPLOIT*
|       FBC8A8BE-F00A-5B6D-832E-F99A72E7A3F7    9.1     https://vulners.com/
B6D-832E-F99A72E7A3F7   *EXPLOIT*
|       E606D7F4-5FA2-5907-B30E-367D6FFECD89    9.1     https://vulners.com/
907-B30E-367D6FFECD89   *EXPLOIT*
|       D8A19443-2A37-5592-8955-F614504AAF45    9.1     https://vulners.com/
592-8955-F614504AAF45   *EXPLOIT*
|       CNVD-2025-16610 9.1     https://vulners.com/cnvd/CNVD-2025-16610
|       CNVD-2024-36387 9.1     https://vulners.com/cnvd/CNVD-2024-36387
|       CNVD-2024-33814 9.1     https://vulners.com/cnvd/CNVD-2024-33814
|       B5E74010-A082-5ECE-AB37-623A5B33FE7D    9.1     https://vulners.com/
```

nmap -sU -p53,161 192.168.0.7

```
~/Documents/IFCT0109 ❯ nmap -sU -p53,161 192.168.0.7
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 10:17 CET
Nmap scan report for 192.168.0.7
Host is up (0.0015s latency).

PORT    STATE  SERVICE
53/udp  closed domain
161/udp closed snmp
MAC Address: 08:00:27:44:AC:A5 (PCS Systemtechnik/Oracle VirtualBox

Nmap done: 1 IP address (1 host up) scanned in 0.26 seconds
```

Ejercicio 4: Comunicación y transferencia de archivos con Netcat

Herramienta: Netcat (nc)
Objetivo: Aprender a usar **Netcat**, una herramienta flexible para abrir conexiones TCP/UDP, escuchar en puertos y transferir datos de forma simple. Se realizarán dos demostraciones: un **chat simple** entre dos máquinas y una **transferencia de archivo**, destacando que Netcat no aplica cifrado ni autenticación (lo cual implica riesgos de seguridad).

Instrucciones:

1. **Chat entre dos VMs:** Supongamos que tenemos dos máquinas en el laboratorio, VM-A y VM-B. En VM-A (que actuará de "servidor"), abra una terminal y ejecute nc -l -p 5000. Esto hace que Netcat **escuche (listen)** en el puerto TCP 5000 en VM-A. Ahora en VM-B, abra otra terminal y conéctese a VM-A ejecutando nc <IP_de_VM-A> 5000. Si todo va bien, la terminal en VM-B se quedará en blanco (ha establecido la conexión).

2. Escriba un mensaje en la terminal de VM-B (por ejemplo: "Hola desde B") y presione Enter. Verifique la terminal de VM-A: debería mostrar el texto enviado por VM-B. Ahora escriba algo en VM-A ("Hola, te escucho") y enter; observe que aparece en VM-B. ¡Acaba de crear un chat rudimentario! Ambas terminales están enviando y recibiendo texto a través de Netcat.

3. **Transferencia de archivo:** Ahora usaremos Netcat para enviar un archivo de VM-B a VM-A. En VM-A, prepárese para recibir creando un comando que vuelque lo recibido a un fichero. Por ejemplo, en VM-A ejecute: nc -l -p 5001 > archivo_recibido.txt. Queda esperando conexiones en el puerto 5001 y redirige todo lo que reciba al fichero *archivo_recibido.txt*.

4. En VM-B, suponga que tenemos un archivo de texto llamado *mensaje.txt* que queremos enviar. Ejecute en VM-B: nc <IP_de_VM-A> 5001 < mensaje.txt. Esto abrirá una conexión a VM-A:5001 y **redirigirá** el contenido de *mensaje.txt* como entrada de Netcat, enviándolo por la red.

5. Verifique en VM-A: cuando el comando de VM-B termine, VM-A regresará al prompt. Abra el fichero *archivo_recibido.txt* en VM-A (por ejemplo con cat archivo_recibido.txt o en un editor) y compruebe que su contenido coincide con *mensaje.txt* de VM-B. ¡Ha transferido un archivo con Netcat!

6. Experimente con otros usos si desea: Netcat también puede usarse para probar puertos (por ejemplo, nc -z -v <IP> 1-100 haría un escaneo de puertos rápido), o para crear un canal *pipe* entre comandos (podría enviar el output de un tar para transferir directorios completos, etc.). Pero estos usos exceden este ejercicio básico.

7. Cierre las conexiones y procesos de Netcat con Ctrl+C cuando haya terminado en cada VM.

Comandos necesarios:
En VM-A (servidor):

```
nc -l -p 5000                  # Escuchar en puerto 5000 para chat

nc -l -p 5001 > archivo_recibido.txt   # Escuchar en 5001 y guardar lo recibido en archivo
```

En VM-B (cliente):

```
nc 192.168.1.5 5000            # Conectarse al chat en VM-A (IP 192.168.1.5) puerto 5000

nc 192.168.1.5 5001 < mensaje.txt     # Enviar archivo mensaje.txt a VM-A:5001
```

Riesgos asociados y mitigación:

- **Riesgo:** Netcat transmite datos **en texto claro** y no requiere autenticación. Cualquiera que intercepte la comunicación (ej. via sniffer) podría leer el contenido. Además, abrir puertos con Netcat podría ser peligroso si se deja abierto inadvertidamente, ya que actuaría como *backdoor* aceptando conexiones.
  Mitigación: No usar Netcat para transferir datos sensibles a menos que esté dentro de un túnel seguro (o usar alternativas seguras como **SSH/SCP/SFTP** que cifran la conexión). Controlar y limitar en el firewall los puertos que se utilizan, y cerrar Netcat inmediatamente después de usarlo. En sistemas de producción, evite tener Netcat instalado o al menos renombre/deshabilite su uso por usuarios no autorizados.

Resultados esperados:

- En la demostración de chat: ambos usuarios podrán enviar y recibir mensajes en tiempo real. Por ejemplo, VM-B envía "Hola desde B" y VM-A lo ve aparecer en su terminal netcat; VM-A responde "Hola, te escucho" y VM-B lo ve en la suya. Esto confirma que Netcat estableció una comunicación bidireccional en el puerto 5000 entre las dos máquinas.
- En la transferencia de archivo: el fichero *archivo_recibido.txt* en VM-A debe tener exactamente los mismos contenidos que *mensaje.txt* en VM-B (puede comparar los hashes MD5 de ambos archivos para estar seguro, deberían coincidir). Netcat no provee confirmación explícita de que el archivo llegó, pero el hecho de que volvió al prompt y el contenido coincide indica una transferencia exitosa.
- No se espera salida en pantalla durante la transferencia (salvo quizás en VM-B si había -v verbose se vería bytes enviados). En el chat, la salida son los mensajes mismos.
- Si alguien estuviera esnifando la red en medio (lo que simularemos en otro ejercicio), vería el texto en claro. Esto refuerza el punto del riesgo.

Solución guiada:
El alumno comprobó que **Netcat es extremadamente versátil** pero también *muy básico* en cuanto a seguridad. En la práctica de chat, se logró una comunicación similar a Telnet:

cualquier texto escrito se envía tal cual. Si se hubiera capturado el tráfico con Wireshark, por ejemplo, se vería exactamente "Hola desde B" atravesando la red, sin cifrado alguno. Este ejercicio demuestra lo que en el mundo UNIX se dice: *Netcat es como la navaja suiza de las redes*, pues con un solo comando pudo crear un servidor de escucha y con otro un cliente que se conecta, habilitando una comunicación arbitraria.

La transferencia de archivos mostró otra faceta útil: redirigir la entrada/salida estándar (>, <) permite usar Netcat como herramienta de copia de archivos entre sistemas. De hecho, se podrían encadenar comandos (p.ej., tar -cf - carpeta | nc ... en un lado, y nc ... | tar -xf - en el otro) para transferir estructuras completas. El alumno vio que *archivo_recibido.txt* es una copia exacta del original, lo cual verifica la integridad básica de la transferencia (no obstante, en un entorno sin errores de red, esta integridad se mantiene, pero no hay sumas de verificación a menos que el usuario las calcule manualmente).

En cuanto a reflexión de seguridad: Netcat **no autentica** las conexiones. En nuestro caso, VM-A escuchaba en 5000/5001 y aceptó *lo que viniera*. Si en lugar de VM-B se hubiese conectado un atacante, VM-A habría aceptado la conexión igualmente. Por eso, en entornos seguros no se debe dejar un Netcat escuchando a menos que sea absolutamente necesario y se tenga control del segmento de red. Asimismo, cualquier firewall perimetral detectaría esta actividad como inusual si los puertos 5000/5001 no están normalmente en uso. En un pentest, los auditores a veces usan Netcat precisamente para crear *backdoors* (ej.: en una máquina comprometida, arrancan nc -l -p 4444 -e /bin/bash para obtener una shell remota). Sin cifrado, eso podría ser detectado, pero es rápido de hacer.

La mitigación mencionada (usar SSH/SFTP en su lugar) es la práctica estándar: si deseamos la misma funcionalidad pero segura, **SSH** ofrece shells remotas cifradas y **SCP/SFTP** transferencia de archivos cifrada. Netcat no cifra, así que todo lo que hicimos fue susceptible de *sniffing*. En ejercicios posteriores, probablemente el alumno comprobará esto capturando tráfico (por ejemplo, con Wireshark, vería el contenido del chat en texto claro).

En resumen, el alumno aprendió a usar Netcat para conexión y transferencia, y entendió por qué es una herramienta potente pero **riesgosa** si se usa sin precaución. Es una excelente utilidad en laboratorio, pero en producción se prefiere limitarla o reemplazarla por alternativas seguras.

nc -l -p 5000

```
~/Documents/IFCT0109 ❯ nc -l -p 5000
hola
```

nc localhost 5000

```
~ ❯ nc localhost 5000
hola
```

nc -lp 5002 > archivo_recibido.txt

cat archivo_recibido.txt

```
~/Documents/IFCT0109 ❯ nc -lp 5002 > archivo_recibido.txt
~/Documents/IFCT0109 ❯ cat archivo_recibido.txt
"hola alumnos"
```

touch mensaje.txt

echo "\"hola alumnos\"" > mensaje.txt

nc localhost 5001 < mensaje.txt

```
~ ❯ touch mensaje.txt
~ ❯ echo "\"hola alumnos\"" > mensaje.txt
~ ❯ nc localhost 5001 < mensaje.txt
```

Ejercicio 5: Captura y análisis de tráfico con Wireshark

Herramienta: Wireshark
Objetivo: Utilizar **Wireshark**, un analizador de protocolos de red gráfico, para capturar paquetes en la red local y analizar su contenido. El alumno aprenderá a filtrar tráfico y a identificar información útil (por ejemplo, visualizar una sesión *HTTP* sin cifrar o un *ping* ICMP) dentro de una captura, evidenciando lo que un atacante o auditor puede observar si tiene acceso a la red.

Instrucciones:

1. En una máquina de laboratorio (idealmente la misma donde se hicieron los ejercicios previos de Netcat y pings), abra **Wireshark**. Seleccione la interfaz de red adecuada (por ejemplo, eth0 si es Ethernet, o wlan0 si es Wi-Fi, etc.).
2. Haga clic en "Start Capturing" (el icono de aleta de tiburón azul) para iniciar la captura de todos los paquetes en esa interfaz. Notará que empiezan a aparecer listados de paquetes en tiempo real.
3. Genere algo de tráfico que podamos analizar:
    - Por ejemplo, **ejecute un ping** desde esta máquina a otra (Ejercicio 1) para capturar ICMP.
    - O realice una pequeña navegación web en HTTP: puede usar un sitio no cifrado como http://neverssl.com (que siempre responde en HTTP plano) para capturar peticiones HTTP.
    - Si realizó el Ejercicio 4 (Netcat chat), incluso puede usarlo: envíe unas líneas por Netcat mientras Wireshark captura para ver esos datos pasar.
4. Detenga la captura tras unos segundos de actividad (botón cuadrado rojo). Ahora tiene un fichero de captura con múltiples paquetes.
5. **Filtrar por protocolo:** En la barra de filtro (arriba de la lista de paquetes), escriba icmp y pulse Enter para ver solo paquetes ICMP. Debería ver sus *Echo (ping) Request* y *Echo Reply*. Haciendo clic en uno, en el panel inferior puede ver detalles: la capa IP, la capa ICMP, etc., incluyendo el *identifier/sequence* del ping y los datos. Esto confirma que Wireshark detectó el ping y muestra su contenido (que es básicamente un saludo o un patrón de bytes).
6. Borre el filtro y ahora filtre por http (si visitó neverssl.com u otro sitio HTTP). Deberían aparecer paquetes HTTP GET y HTTP responses. Busque la primera que diga "GET / … HTTP/1.1". Al seleccionarla, en el panel inferior verá la cabecera HTTP (HOST, User-Agent, etc.) y puede que hasta contenido si la página es simple. Compruebe en texto si aparecen cosas como <html> indicando el contenido web recibido.

7. Otra forma: haga clic derecho en un paquete HTTP y elija "Follow > HTTP Stream". Wireshark mostrará en una ventana aparte la conversación completa HTTP entre su PC y el servidor, reconstruyendo el texto (página HTML, etc.). Notará que cualquier cosa, incluyendo quizás credenciales si las hubiera en un formulario sin cifrar, aparecería aquí.
8. Si capturó el chat de Netcat (puerto 5000 del Ejercicio 4): use un filtro tcp.port == 5000. Aparecerán los paquetes de la sesión. Click derecho en uno y "Follow > TCP Stream". Verá el diálogo de chat tal cual los escribieron, en texto claro.
9. Experimente con la interfaz: Wireshark permite ver **estadísticas** (menú Statistics) como "Protocol Hierarchy" para ver porcentajes de tipos de tráfico, o "Endpoints" para ver las IPs involucradas. Revise por curiosidad.
10. Cierre Wireshark cuando termine (puede guardar la captura si lo desea para reportarla).

Comandos necesarios:
(Wireshark es gráfico, pero se puede ejecutar tshark equivalente en terminal. Sin embargo, aquí asumimos GUI.)

- Uso de filtros (en la caja de filtrado de Wireshark, no en terminal):
  - Filtro ICMP: icmp
  - Filtro HTTP: http
  - Filtro por puerto específico: tcp.port == 5000

Riesgos asociados y mitigación:

- **Riesgo:** Wireshark en sí no introduce riesgo, pero **capturar tráfico** muestra cuánta información sensible viaja en claro. Un atacante con capacidad de sniffing podría leer contraseñas, correos, webs visitadas si no están cifradas.
  Mitigación: Cifrar las comunicaciones es fundamental. Usar HTTPS en lugar de HTTP, SSH en lugar de Telnet/Netcat, VPN en redes Wi-Fi, etc., de forma que aunque alguien capture paquetes, vea datos incomprensibles. Además, segmentar la red y aplicar filtros (por ejemplo, evitar que interfaces en modo promíscuo capturen en switches mediante seguridad de puertos) dificulta a atacantes internos.

Resultados esperados:

- El alumno debería ver claramente ejemplos de protocolos en acción:
  - Para el **ping**: en el panel central se listarán paquetes "Echo (ping) request" y "Echo reply". En el panel inferior, al abrir ICMP, verá campos como *Type: 8 (Echo)* y *Type: 0 (Echo Reply)*, identificadores y la carga de datos (muchas veces son bytes como abcd...). Esto demuestra cómo se ve un ping en la red.

    - Para la **navegación HTTP**: Debería encontrar la petición GET que su navegador hizo. Por ejemplo: *GET /index.html HTTP/1.1\r\n Host: neverssl.com\r\n …* y la respuesta del servidor con el código 200 y contenido HTML. En texto plano podría incluso leer porciones de la página devuelta (por ejemplo, el título de la página). Si hubiera enviado credenciales en un formulario de login HTTP (no fue el caso con neverssl, pero suponga un sitio con auth básica), esas credenciales aparecerían codificadas en Base64 en una cabecera *Authorization:* que se puede decodificar fácilmente.
    - Para el **chat de Netcat**: Al seguir el stream TCP, verá exactamente las frases que se enviaron, delimitadas por quien las envió (Wireshark colorea de rojo vs azul las direcciones origen/destino). Esto confirma que cualquier texto transmitido sin cifrar es recuperable por un analizador.
- Estadísticas: Por ejemplo, si solo hizo un poco de ping y HTTP, *Protocol Hierarchy* mostrará X% ICMP, Y% HTTP, etc. No es central al ejercicio, pero le da idea del mix de tráfico.

Solución guiada:
Wireshark habrá permitido al alumno **visualizar el tráfico de la red a nivel detallado**. Esto solidifica muchos conceptos: que un *ping* no es mágico, sino que son paquetes ICMP viajando con un identificador y unos datos; que acceder a una web por HTTP envía peticiones con cabeceras legibles; que su chat de Netcat efectivamente mandó los caracteres de forma entendible por cualquiera con posición de espía en la red.

Por ejemplo, en la captura HTTP el alumno posiblemente vio en texto claro la frase *"Welcome to NeverSSL"*, confirmando que la página viajó sin cifrar. De la misma manera, si hubiese capturado una sesión de inicio de sesión en un sitio sin TLS, hubiera podido ver algo como *"username=admin&password=12345"* dentro de un paquete, lo cual es alarmante desde el punto de vista de seguridad.

Este ejercicio demuestra el **riesgo de protocolos no cifrados**. En auditorías, usar Wireshark (u otras herramientas CLI como tcpdump) es rutinario para inspeccionar tráfico y detectar, por ejemplo, si cierta aplicación manda contraseñas en texto plano. De hecho, Wireshark facilitó encontrar información sensible en algunos de los protocolos del curso: por eso insistimos en mitigaciones como forzar HTTPS o usar TLS en correos, etc. Un atacante interno conectado a la misma red podría lanzar Wireshark (o un sniffer similar) y recolectar información valiosa si la red está abierta o conmutada sin protecciones.

El alumno también aprendió el uso de **filtros** para aislar lo relevante de entre cientos de paquetes. Filtrar por protocolo o puerto es esencial para centrarse en lo que buscamos. Por ejemplo, en una captura grande de una oficina, uno podría filtrar http && ip.addr == 192.168.1.50 para ver solo tráfico web de un host particular, etc. Wireshark es poderoso en este análisis.

Como conclusión, Wireshark confirmó de manera práctica que toda comunicación no cifrada es **transparente** para quien la capture. La contramedida clara es cifrar: si repetimos el experimento visitando un sitio HTTPS, Wireshark no mostraría el contenido de la página (veríamos TLS handshake, datos cifrados que no podemos leer). Igual con SSH en lugar de Telnet: veríamos garble. Por lo tanto, el ejercicio refuerza la importancia de las mejores prácticas de seguridad en red. Además, demostró una habilidad técnica clave: **inspeccionar y entender el tráfico de red**, algo que un profesional de seguridad debe dominar.

icmpv6

icmpv6

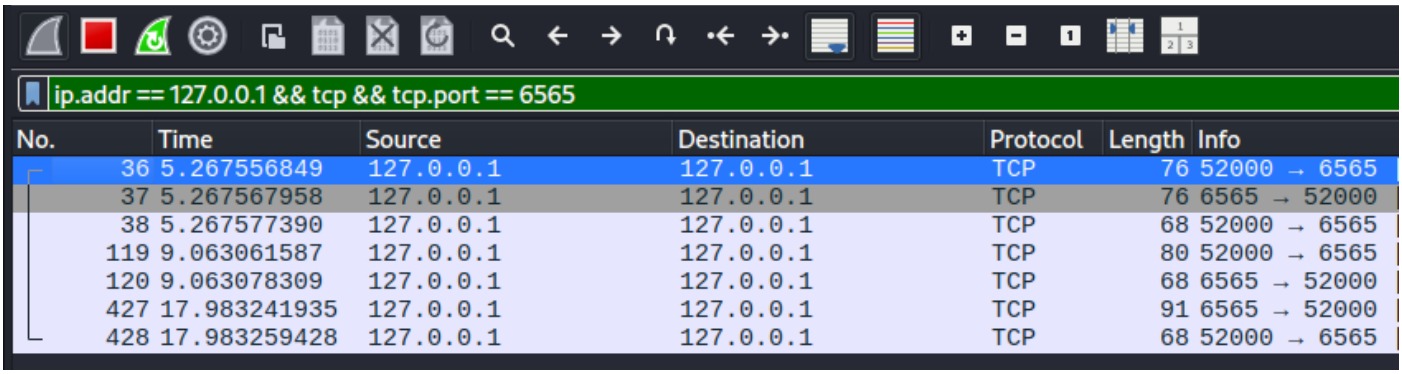
No.	Time	Source	Destination	Protocol
45	9.279803520	::1	::1	ICMPv6…
46	9.279810223	::1	::1	ICMPv6…
50	10.298827234	::1	::1	ICMPv6…
51	10.298833509	::1	::1	ICMPv6…
53	11.300470506	::1	::1	ICMPv6…
54	11.300479263	::1	::1	ICMPv6…
55	12.307757456	::1	::1	ICMPv6…
56	12.307772137	::1	::1	ICMPv6…
66	13.330773562	::1	::1	ICMPv6…
67	13.330779490	::1	::1	ICMPv6…
71	14.359447571	::1	::1	ICMPv6…
72	14.359459659	::1	::1	ICMPv6…

http

http

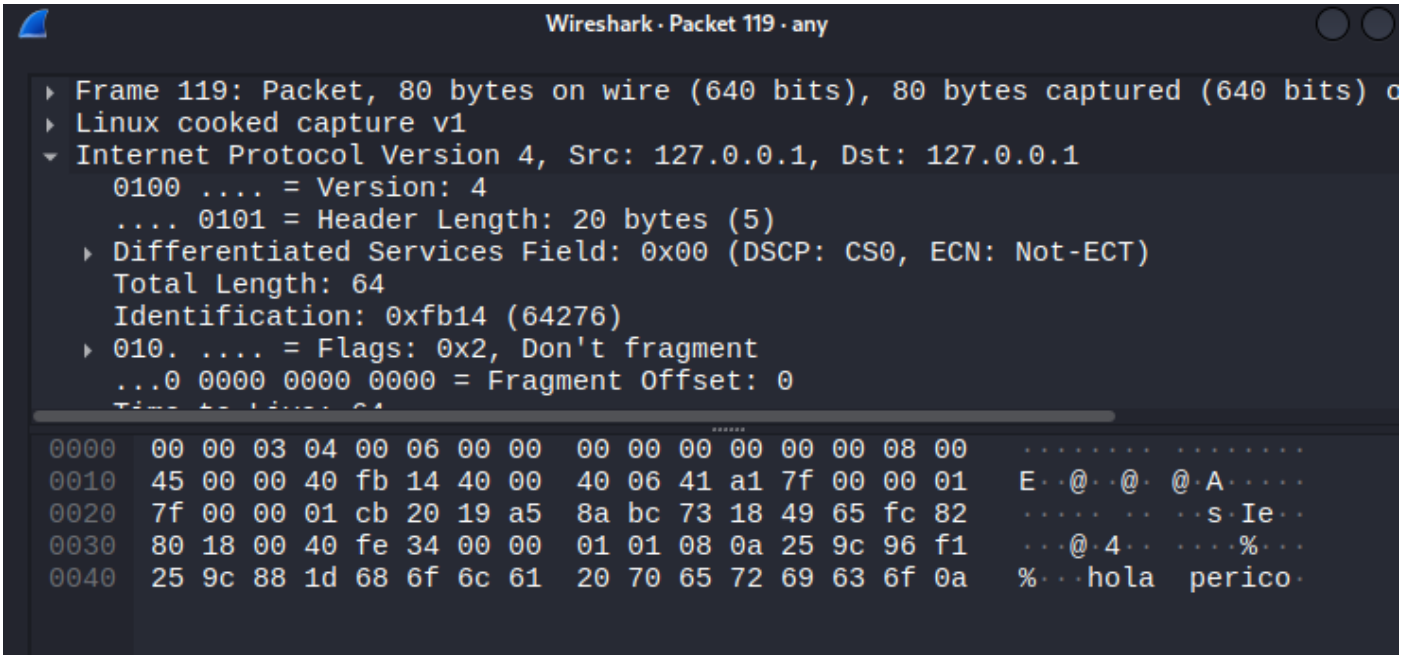
No.	Time	Source	Destination	Protocol	Length	Info
81	4.719656345	172.18.0.4	172.18.0.5	HTTP	669	GET /
95	4.722388738	172.18.0.5	172.18.0.4	HTTP	598	HTTP/
132	4.822389115	172.18.0.4	172.18.0.5	HTTP	751	GET /
136	4.823144731	172.18.0.5	172.18.0.4	HTTP	305	HTTP/
159	4.831588440	172.18.0.4	172.18.0.5	HTTP	749	GET /
175	4.834176648	172.18.0.4	172.18.0.5	HTTP	731	GET /
179	4.835086922	172.18.0.5	172.18.0.4	HTTP	305	HTTP/
192	4.836751753	172.18.0.5	172.18.0.4	HTTP	306	HTTP/
222	4.935655055	172.18.0.4	172.18.0.5	HTTP	708	GET /
241	4.939371445	172.18.0.5	172.18.0.4	HTTP/J…	69	HTTP/
279	5.022938026	172.18.0.4	172.18.0.5	HTTP/J…	750	POST
297	5.026224642	172.18.0.5	172.18.0.4	HTTP/J…	69	HTTP/
324	5.083936914	172.18.0.4	172.18.0.5	HTTP	666	GET /
346	5.086209997	172.18.0.5	172.18.0.4	HTTP/J…	69	HTTP/

ip.addr == 127.0.0.1 && tcp && tcp.port == 6565



ip.addr == 127.0.0.1 && tcp && tcp.port == 6565

No.	Time	Source	Destination	Protocol	Length	Info
36	5.267556849	127.0.0.1	127.0.0.1	TCP	76	52000 → 6565
37	5.267567958	127.0.0.1	127.0.0.1	TCP	76	6565 → 52000
38	5.267577390	127.0.0.1	127.0.0.1	TCP	68	52000 → 6565
119	9.063061587	127.0.0.1	127.0.0.1	TCP	80	52000 → 6565
120	9.063078309	127.0.0.1	127.0.0.1	TCP	68	6565 → 52000
427	17.983241935	127.0.0.1	127.0.0.1	TCP	91	6565 → 52000
428	17.983259428	127.0.0.1	127.0.0.1	TCP	68	52000 → 6565



Wireshark · Packet 119 · any

```
▸ Frame 119: Packet, 80 bytes on wire (640 bits), 80 bytes captured (640 bits) o
▸ Linux cooked capture v1
▾ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 64
    Identification: 0xfb14 (64276)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0

0000  00 00 03 04 00 06 00 00  00 00 00 00 00 00 08 00   ........ ........
0010  45 00 00 40 fb 14 40 00  40 06 41 a1 7f 00 00 01   E··@··@· @·A·····
0020  7f 00 00 01 cb 20 19 a5  8a bc 73 18 49 65 fc 82   ····· ·· ··s·Ie··
0030  80 18 00 40 fe 34 00 00  01 01 08 0a 25 9c 96 f1   ···@·4·· ····%···
0040  25 9c 88 1d 68 6f 6c 61  20 70 65 72 69 63 6f 0a   %···hola  perico·
```

Ejercicio 6: Ataque Man-in-the-Middle en LAN con ARP Spoofing (arpspoof)

Herramienta: arpspoof (suite dsniff)
Objetivo: Practicar un ataque de **ARP Spoofing** para posicionarse como intermediario (*Man-in-the-Middle*, MitM) en una red local. Usaremos *arpspoof* para envenenar la caché ARP de un cliente y del gateway, de modo que el tráfico de la víctima pase a través del atacante. Este ejercicio permite entender la vulnerabilidad del protocolo ARP (no autenticado) y cómo un atacante puede interceptar comunicaciones en LAN. *(Este ataque debe realizarse **solo en laboratorio** controlado, ya que puede afectar la red.)*

Instrucciones:

1. **Preparar entorno:** Necesitamos tres roles: un **Atacante** (con Kali Linux u otra distro con *arpspoof*), una **Víctima** (por ejemplo, un PC Windows/Linux objetivo) y el **Gateway** (puerta de enlace/router de la red). Anote las IP de la víctima y del gateway. Supongamos Atacante=192.168.1.100, Víctima=192.168.1.50, Gateway=192.168.1.1.

2. En la máquina Atacante, habilite el reenvío de paquetes IP para que actúe de puente. En Linux: echo 1 > /proc/sys/net/ipv4/ip_forward. Sin esto, el ataque MitM cortaría la conexión (porque los paquetes llegarían al atacante pero no seguirían al destino).

3. **Envenenamiento ARP (víctima -> atacante):** Abra una terminal (como root) en Atacante y ejecute: arpspoof -t 192.168.1.50 192.168.1.1. Esto envía respuestas ARP falsas a la Víctima diciéndole "la IP 192.168.1.1 (gateway) está en la MAC del Atacante". Mantenga esta terminal abierta; *arpspoof* seguirá enviando paquetes periódicamente.

4. **Envenenamiento ARP (gateway -> atacante):** En otra terminal del Atacante, ejecute: arpspoof -t 192.168.1.1 192.168.1.50. Esto hace lo simétrico: engaña al Gateway diciéndole que "192.168.1.50 (víctima) está en la MAC del Atacante". Ahora el Atacante está al medio de ambas.

5. **Verificación:** En la máquina Víctima, abra una consola y use arp -a (Windows) o ip neigh (Linux) para ver la tabla ARP. Debería observar que la entrada para 192.168.1.1 (gateway) tiene ahora la dirección MAC del Atacante (en lugar de la real del router). Igualmente, si revisa la ARP del router (requiere acceso al router o un tcpdump allí), la MAC para .50 será la del atacante. Esto confirma el spoof.

6. Ahora, genere tráfico desde la Víctima hacia internet o cualquier recurso (por ejemplo, abra una página web o continúe con el ping hacia el gateway). El Atacante, teniendo IP forwarding activo, reenviará ese tráfico al destino real. Para ver que está ocurriendo, en el Atacante abra Wireshark y capture; debería ver tanto paquetes desde Víctima a Atacante a Gateway, como de Gateway a Atacante a Víctima. El atacante puede ver (y si quisiera modificar) estos paquetes.

7. (Opcional) Pruebe la navegación web desde la Víctima. Mientras *arpspoof* corre, en la Víctima abra un sitio web (HTTP preferiblemente, para observar contenido). En Atacante, la captura de Wireshark mostrará las peticiones que hace la Víctima y las respuestas. Sin el ataque, el Atacante no vería nada de tráfico ajeno; con el ataque, se convierte en un sniffer efectivo aunque la red use switch.
8. **Limpiar:** Pare los procesos *arpspoof* (Ctrl+C en sus terminales). Vuelva a deshabilitar el forwarding si desea (echo 0 > /proc/sys/net/ipv4/ip_forward). La caché ARP de la Víctima eventualmente expirará o puede recuperarla reiniciando la conexión de red o haciendo arp -d para borrar entradas.

Comandos necesarios:
En máquina Atacante (Linux):

```
echo 1 > /proc/sys/net/ipv4/ip_forward          # Habilitar reenvío IP
arpspoof -t 192.168.1.50 192.168.1.1            # Spoofear Victima diciéndole que Gateway=MAC_atacante
arpspoof -t 192.168.1.1 192.168.1.50            # Spoofear Gateway diciéndole que Victima=MAC_atacante
echo 0 > /proc/sys/net/ipv4/ip_forward          # (Después) Deshabilitar reenvío IP
```

En máquina Víctima (para verificar ARP):

```
arp -a          # (Windows) Ver tabla ARP
ip neigh show   # (Linux) Ver vecinos ARP
```

Riesgos asociados y mitigación:

- **Riesgo:** Este ataque permite la **interceptación** o manipulación de tráfico de red, comprometiendo potencialmente información sensible (credenciales, sesiones) y pudiendo causar interrupciones (si el atacante falla en reenviar, hay DoS). Es muy peligroso en redes locales abiertas (como Wi-Fi público) donde cualquiera puede aplicarlo.
  Mitigación: Implementar defensas a nivel de red: por ejemplo **entradas ARP estáticas** en servidores críticos (así ignoran ARP spoof), usar **ARP Spoofing Detection** en switches o IPS, protocolos como **DAI (Dynamic ARP Inspection)** en switches gestionados, o utilizar conmutadores con seguridad de puertos para evitar que un puerto envíe muchas respuestas ARP falsas. A nivel general, emplear cifrado de extremo a extremo (HTTPS, SSH) mitiga el impacto (el atacante puede ver tráfico pero cifrado). Redes segmentadas y monitoreadas (un IDS puede alertar de ARP anómalos) también ayudan.

Resultados esperados:

- Tras ejecutar arpspoof, la **tabla ARP de la Víctima** mostrará la MAC del Atacante asociada a la IP del Gateway. Por ejemplo, antes del ataque podía ser *192.168.1.1 at aa:bb:cc:dd:ee:ff* (MAC del router), y durante el ataque se verá *192.168.1.1 at 11:22:33:44:55:66* (MAC del atacante). Esto indica que la Víctima ahora envía sus paquetes destinados al gateway hacia el atacante.
- En la **tabla ARP del Gateway** (si se pudiera ver) pasaría lo inverso para la IP de la Víctima.
- La **Víctima en principio no notará nada extraño**: la navegación web, ping, etc., seguirán funcionando (quizá con una leve latencia adicional), ya que el Atacante está reenviando el tráfico. Esto es clave: el ataque es silencioso desde la perspectiva del usuario.
- En el Atacante, Wireshark o tcpdump mostrará que está viendo tráfico cuyo origen o destino es la Víctima cuando no debería normalmente. Por ejemplo, el Atacante verá peticiones DNS de la Víctima, o HTTP, etc., pasando por él.
- Si el Atacante quisiera, podría **modificar** los paquetes (no lo hicimos aquí). Por ejemplo, podría filtrar y alterar una página HTTP en tránsito (haciendo de proxy). No obstante, aun sin modificar nada, ya tiene la capacidad de leer o registrar todo.

Solución guiada:
El ejercicio muestra en la práctica cómo un atacante puede convertise en "hombre en el medio" en una LAN con conmutador, explotando la confianza ciega del protocolo ARP. El alumno habrá visto que, tras el ataque, cualquier comunicación entre la Víctima y su gateway (Internet) **pasa ahora por el Atacante**. Esto significa que el atacante puede espiar o incluso inyectar tráfico.

Por ejemplo, supongamos que durante el ataque la Víctima inició sesión en un sitio web sin HTTPS. El Atacante, ejecutando Wireshark, habría capturado las credenciales en texto claro. Incluso con HTTPS, si el Atacante realiza técnicas avanzadas (como spoofing DNS combinado con servidor falso) podría engañar a la Víctima a ir a un sitio falso - este es el principio de ataques de phishing en LAN. El ejercicio se limitó a ARP, pero sienta las bases para esos escenarios.

La mitigación explica que existen formas de endurecer la red: **entradas ARP estáticas** (por ejemplo, fijar la MAC del gateway en todos los hosts, así ignoran anuncios contradictorios) previene este ataque, aunque es difícil de mantener a gran escala. Los switches modernos pueden detectar inundaciones ARP o discrepancias (DAI, etc.) y bloquear al atacante. También se promueve el uso de **VPN en Wi-Fi públicas** porque en redes abiertas es trivial hacer ARP spoof y espiar a otros; con una VPN, aunque intercepten, todo va cifrado. Además,

herramientas de monitoreo pueden detectar patrones de ARP inusuales (por ejemplo, un IDS puede alertar si una IP cambia de MAC demasiado rápido).

El alumno, con la verificación arp -a, vio directamente que la **resolución de IP->MAC fue alterada**. Este es un síntoma claro del ataque. En entornos empresariales, si un administrador ve ARP cache inconsistente, sabe que algo anda mal.

Es importante destacar la **naturaleza temporal** del envenenamiento: al detener arpspoof, después de unos minutos la caché ARP de la Víctima olvidará la entrada falsa y volverá a pedir la MAC real, restaurando la normalidad. Pero en el intervalo, el atacante tuvo vía libre.

Con este ejercicio, el alumno comprende un método clave de ataques internos y la razón por la cual protocolos antiguos y sin cifrado son vulnerables en redes locales comprometidas. También prepara el terreno para usar otras herramientas de la suite dsniff (como *dnsspoof*, *mailsnarf*, etc.), que suelen requerir primero situarse en MitM mediante ARP spoofing, justo lo que logramos aquí.

sudo arpspoof -t 192.168.0.7 192.168.0.1

```
~/Documents/box ❯ sudo arpspoof -t 192.168.0.7 192.168.0.1
[sudo] password for kali:
8:0:27:d1:f8:5d 8:0:27:44:ac:a5 0806 42: arp reply 192.168.0.1 is-at 8:0:27:d1:f8:5d
8:0:27:d1:f8:5d 8:0:27:44:ac:a5 0806 42: arp reply 192.168.0.1 is-at 8:0:27:d1:f8:5d
8:0:27:d1:f8:5d 8:0:27:44:ac:a5 0806 42: arp reply 192.168.0.1 is-at 8:0:27:d1:f8:5d
8:0:27:d1:f8:5d 8:0:27:44:ac:a5 0806 42: arp reply 192.168.0.1 is-at 8:0:27:d1:f8:5d
```

arp -a

```
~ ❯ arp -a
? (192.168.0.116) at 96:d4:93:98:92:f0 [ether] on eth0
? (192.168.0.96) at d8:43:ae:44:e2:69 [ether] on eth0
? (192.168.0.37) at f4:1a:9c:16:51:18 [ether] on eth0
? (192.168.0.81) at 08:00:27:c7:24:88 [ether] on eth0
? (192.168.0.22) at 08:00:27:cb:65:ff [ether] on eth0
? (192.168.0.132) at d8:43:ae:44:e2:72 [ether] on eth0
? (192.168.0.73) at 4a:fd:90:1d:b4:5c [ether] on eth0
? (192.168.0.117) at 98:ee:cb:46:94:f7 [ether] on eth0
? (192.168.0.94) at 08:00:27:1f:b7:23 [ether] on eth0
? (192.168.0.133) at f8:54:f6:bb:26:70 [ether] on eth0
? (192.168.0.74) at 56:a8:87:80:32:6c [ether] on eth0
? (192.168.0.110) at 2c:41:38:ac:7d:1c [ether] on eth0
? (192.168.0.28) at 78:ac:c0:c2:c6:66 [ether] on eth0
? (192.168.0.134) at 14:13:33:e5:20:c1 [ether] on eth0
? (192.168.0.32) at c0:14:3d:c0:19:e9 [ether] on eth0
? (192.168.0.111) at cc:28:aa:c9:15:46 [ether] on eth0
```

ip neigh show

```
~ ❯ ip neigh show
192.168.0.116 dev eth0 lladdr 96:d4:93:98:92:f0 STALE
192.168.0.96 dev eth0 lladdr d8:43:ae:44:e2:69 STALE
192.168.0.37 dev eth0 lladdr f4:1a:9c:16:51:18 STALE
192.168.0.81 dev eth0 lladdr 08:00:27:c7:24:88 STALE
192.168.0.22 dev eth0 lladdr 08:00:27:cb:65:ff STALE
192.168.0.132 dev eth0 lladdr d8:43:ae:44:e2:72 STALE
192.168.0.73 dev eth0 lladdr 4a:fd:90:1d:b4:5c STALE
192.168.0.117 dev eth0 lladdr 98:ee:cb:46:94:f7 STALE
192.168.0.94 dev eth0 lladdr 08:00:27:1f:b7:23 STALE
192.168.0.133 dev eth0 lladdr f8:54:f6:bb:26:70 STALE
192.168.0.74 dev eth0 lladdr 56:a8:87:80:32:6c STALE
192.168.0.110 dev eth0 lladdr 2c:41:38:ac:7d:1c STALE
192.168.0.28 dev eth0 lladdr 78:ac:c0:c2:c6:66 STALE
192.168.0.134 dev eth0 lladdr 14:13:33:e5:20:c1 STALE
192.168.0.32 dev eth0 lladdr c0:14:3d:c0:19:e9 STALE
192.168.0.111 dev eth0 lladdr cc:28:aa:c9:15:46 STALE
```

ip.addr == 192.168.0.7

No.	Time	Source	Destination	Protocol	Le
4	0.369629233	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
5	0.371505894	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TLSv1.2	
7	0.591465673	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
8	0.596421634	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
13	0.811151740	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
14	0.827041160	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
15	1.263461208	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
16	1.283057196	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
23	2.159196054	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
24	2.242950766	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
30	3.918680338	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	
31	4.099385585	192.168.0.7	93.243.107.34.bc.googleusercontent.com	TCP	

Ejercicio 7: Interceptación de credenciales en texto claro con DSniff

Herramienta: dsniff
Objetivo: Utilizar **dsniff** para capturar credenciales (usuarios y contraseñas) que circulen sin cifrar por la red. Dsniff es una herramienta que escucha el tráfico en busca de *patrones conocidos* de autenticación en protocolos como HTTP, FTP, SMTP, Telnet, etc., extrayendo las credenciales automáticamente. Este ejercicio asume que ya tenemos capacidad de sniffing (por ejemplo, tras un ARP Spoof exitoso del ejercicio anterior, o ejecutándolo en un segmento hub/mixto). El objetivo es evidenciar qué fácil es obtener contraseñas cuando se usan protocolos no seguros.

Instrucciones:

1. **Preparar entorno:** El *Atacante* debe estar en posición de ver el tráfico de la *Víctima*. Puede ser mediante el ARP spoofing del ejercicio 6 (manténgalo activo si desea, con IP forwarding) o simplemente ejecutando dsniff en una red hub o en la misma máquina donde corren servicios. Asumiremos la configuración de MitM lograda con arpspoof: Atacante sigue en medio de Víctima y Gateway.

2. En la máquina Atacante, ejecute dsniff -i <interface> (por ejemplo, dsniff -i eth0). Este comando comienza a husmear en la interfaz de red indicada. Dsniff no necesita saber qué buscar; internamente tiene firmas para detectar, por ejemplo, cuando ve USER/PASS en FTP, o Authorization: Basic … en HTTP, o credenciales de POP3/IMAP, etc., las registra.

3. Ahora, genere tráfico de la Víctima que contenga una autenticación *no cifrada*. Pueden hacerse varias cosas (no es necesario hacer todas, con una basta para demostrar):

    - **FTP:** Desde la Víctima, conecte a un servidor FTP sin cifrado. Si tiene un servidor FTP en la red (por ejemplo, vsftpd en otra VM), haga ftp <IP_servidor> y cuando pida user/pass, ingrese credenciales de prueba (p. ej. *usuario: test, pass: 12345*).

    - **HTTP autenticación básica:** Navegue desde la Víctima a una URL que requiera Basic Auth. Si no tiene una a mano, puede simularlo con un servidor web configurado con auth básica en el lab. (Otra opción: usar *wget* con credenciales en la URL: wget http://user:pass@<IP_servidor>/pagina provoca envío de auth básica).

    - **Telnet:** Si en otra máquina hay un servicio telnet, intente hacer telnet desde Víctima; las credenciales que se envían (username/password) van en claro.

    - **SMTP/POP3:** Si dispone de un servidor de correo de prueba en la red (por ejemplo, postfix sin TLS, o un hmailserver en Windows), intente desde Víctima enviar un correo (SMTP) o recuperar (POP3) con credenciales simples.

4. Cualquiera de las acciones anteriores generará tráfico con credenciales en texto claro. Vaya a la consola del Atacante donde corre dsniff: debería aparecer una línea por cada conjunto de credenciales detectado. Por ejemplo, al hacer FTP, dsniff puede mostrar: ftp 192.168.1.50:1025 -> 192.168.1.60:21 USER test PASS 12345 indicando que capturó un login FTP con esas credenciales. O para HTTP Basic: http 192.168.1.50 -> 192.168.1.60 auth: Basic dXNlcjpwYXNzd29yZA== (dsniff podría decodificarlo y mostrar user:password).
5. Si no ve nada inmediatamente, verifique que dsniff sigue corriendo y que efectivamente el tráfico de la víctima pasa por el atacante. Tal vez necesite reiterar la acción (por ejemplo, volver a conectar FTP). Asegúrese de que los protocolos no estén usando cifrado (FTP normal, no SFTP; HTTP, no HTTPS).
6. Una vez obtenidas credenciales, puede detener dsniff (Ctrl+C).

Comandos necesarios:
En Atacante:

```
dsniff -i eth0        # Escuchar en interfaz eth0 en busca de credenciales
```

En Víctima (ejemplos de generación de tráfico con credenciales):

```
ftp 192.168.1.60            # Conectar a servidor FTP y luego introducir usuario/clave
telnet 192.168.1.61          # Conectar a servidor Telnet, luego ingresar credenciales
# HTTP Basic Auth via wget (ejemplo):
wget http://usuario:clave@localhost/privado/secret.txt
```

Riesgos asociados y mitigación:

- **Riesgo:** La captura mostró credenciales **en texto plano** circulando por la red. Esto revela que protocolos antiguos (FTP, Telnet, HTTP básico, POP3/IMAP sin SSL) son susceptibles a robo de contraseñas por cualquier atacante con capacidad de sniffing. **Mitigación: No usar protocolos sin cifrar para autenticación.** Emplear siempre las versiones seguras: SFTP/SCP en vez de FTP, SSH en vez de Telnet, HTTPS en vez de HTTP (o al menos HTTP Digest/Auth con TLS), y SMTPS/POP3S/IMAPS para correo. Además, segmentar la red y usar conmutadores con inspección ARP reduce la posibilidad de sniffing, pero la solución de fondo es el cifrado punto a punto.

Resultados esperados:

- En la salida de dsniff, cada vez que detecte un *login* transmisible, lo imprimirá. Ejemplos concretos que podría ver:

    - Para FTP: ftp [target_ip]:21- <user> <password> como mencionamos (dsniff identifica los comandos FTP USER y PASS).
    - Para HTTP Basic: una línea http con la credencial. Dsniff a veces muestra la credencial decodificada directamente en formato username/password después de "auth:". Si no, presentaría la string base64 que contiene "user:pass" (que es fácilmente decodificable).
    - Para POP3: podría ver pop <src_ip>:<port> -> <dst_ip>:110 USER alice PASS wonderland.
    - Para SMTP (si el cliente realiza AUTH LOGIN, común en SMTP sin TLS): dsniff decodificaría el Base64 y mostraría el usuario y pass del correo.
    - Para Telnet: Telnet es más difícil porque envía caracteres a medida que se escriben, pero dsniff puede detectar la secuencia típica de login. A veces no pilla telnet user/pass, pero Wireshark sí lo haría. Centrémonos en los casos que dsniff cubre bien (FTP/HTTP/POP).
- Lo importante es que **al menos una credencial** con usuario y contraseña en texto claro sea mostrada en la terminal del Atacante. Eso es la evidencia de éxito.
- La Víctima no se entera de nada; sus sesiones de FTP o web funcionan igual. Solo que el atacante obtuvo las claves en paralelo.

Solución guiada:
El alumno habrá obtenido, con asombrosa facilidad, credenciales ajenas mediante dsniff. Por ejemplo, habrá visto algo como "*ftp credential captured: user=test, pass=12345*" impreso en pantalla, confirmando que la cuenta de prueba de FTP fue comprometida. Esto recalca la gravedad de usar protocolos no cifrados en cualquier entorno donde pudiera haber un atacante presente (lo cual en redes abiertas o comprometidas es siempre una posibilidad).

Al combinar este ejercicio con el anterior: primero se hizo ARP Spoof para interceptar tráfico, luego con dsniff se extrajeron credenciales. Este es exactamente el flujo de ataque de un *sniffer* en LAN. La suite dsniff está diseñada para ser usada así, por lo que el alumno ha recreado un escenario realista de intrusión interna.

Las mitigaciones son claras: **cifrado y protocolos seguros**. En entornos modernos, es inaceptable usar Telnet o FTP para administrar sistemas precisamente por esto. De hecho, la mayoría de organizaciones ya migraron a SSH y SFTP, pero es posible que algunos servicios (impresoras con telnet, cámaras IP, etc.) aún usen credenciales en claro. Un auditor debe detectarlos para suplantar o mitigar.

Además de cifrado, se pudo mencionar autenticación de nivel de red (por ejemplo, redes conmutadas con port security, para que un atacante no pueda fácilmente unirse o crear

MitM). Pero incluso con eso, un empleado malicioso dentro de la subred podría intentar sniffing. Por tanto, la **mejor práctica universal es cifrar** la capa de aplicación.

El ejercicio demuestra de forma impactante que contraseñas triviales pueden recolectarse sin siquiera atacar directamente la máquina víctima, **sino escuchando su tráfico**. El alumno debe reflexionar: cualquier contraseña vista por dsniff equivaldría a que un atacante la obtiene y posteriormente puede reutilizarla para acceso directo (por ejemplo, la cuenta FTP capturada podría usarse para entrar al servidor FTP con total privilegio de ese usuario).

En una auditoría, tras esta fase, se recomendaría: "Deshabilitar servicios FTP no seguros o reemplazarlos por SFTP; Enforce TLS en servidores de correo; Obligar uso de HTTP seguro, etc." justamente porque hemos comprobado la vulnerabilidad.

Cabe mencionar que dsniff no captura credenciales de protocolos cifrados (no verá nada de SSH, HTTPS, etc., más que datos incomprensibles). Esto refuerza el punto: **la criptografía frustra completamente este tipo de ataque**. Por eso la migración a TLS en todo (HTTPS, SMTPS, etc.) es tan crucial.

El alumno sale de este ejercicio con una poderosa lección: a nivel de red local, sin las protecciones adecuadas, **no hay privacidad**. Herramientas como dsniff (o incluso Wireshark manualmente) hacen trivial obtener secretos. Las contramedidas no son opcionales, son obligatorias para garantizar la seguridad básica de las comunicaciones.

sudo dsniff -i eth0

```
~ ❯ sudo dsniff -i eth0
dsniff: listening on eth0
-----------------
11/20/25 10:50:56 tcp 192.168.0.10.39230 -> 194.108.117.16.21 (ftp)
USER demo
PASS password

-----------------
11/20/25 10:52:25 tcp 192.168.0.10.37864 -> archive.scene.org.21 (ftp)
USER ftp
PASS mail@example.com
```

ftp ftp.scene.org

```
~/Documents/IFCT0109 ❯ ftp ftp.scene.org
Connected to archive.scene.org.
220 ftp.scene.org FTP server (SceneOrgFTPD-2.4.4) ready.
Name (ftp.scene.org:kali): ftp
331 Anonymous login ok, send your complete email address as your password
Password:
230-

   _________ _________   __________  _______    __________
  /  ______/ \_   ___ \  \_   ____/  \      \   \_   ____/
  \_____  \  /    \  \/   |    _)_   /   |   \   |    _)_
  /        \ \     \____  |       \ /    |    \  |       \
 /_______  /  \_______  / /______  / \____|__  / /______  /
         \/           \/         \/          \/         \/  . ORG!
 'elektronik free art' since 11 March 1996
                                                            Problems? Mail:
                                                            ftp@scene.org
230 Anonymous access granted, restrictions apply
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||53667|)
150 Opening BINARY mode data connection for file list
```

Ejercicio 8: Suplantación de DNS en LAN con dnsspoof

Herramienta: dnspoof (suite dsniff)
Objetivo: Simular un ataque de **DNS Spoofing** local, donde el atacante responde falsas resoluciones DNS a la víctima, redirigiendo a un dominio a un IP distinto (controlado por el atacante). Esto se usa en ataques para llevar a las víctimas a sitios falsos (phishing) pese a que escriban la URL correcta. En el laboratorio, realizaremos una prueba controlada: configuraremos dnsspoof para que, cuando la víctima intente resolver un nombre específico, el atacante responda con la IP de un servidor falso.

Instrucciones:

1. Asumimos nuevamente el escenario de MitM logrado (Atacante ya en posición vía ARP spoof). También es posible hacer esto sin MitM si la víctima utiliza al atacante directamente como DNS (por ejemplo, si configuramos la IP del atacante como servidor DNS en la víctima). Pero continuemos con MitM: interceptaremos las consultas DNS hacia el DNS real.
2. Cree un archivo de **hosts** personalizado en la máquina Atacante que mapee el nombre de dominio a su IP falsificada. Por ejemplo, cree dnsfake.txt con el contenido:
3. 192.168.1.100 www.ejemplo.com

Suponiendo que 192.168.1.100 es una IP donde el atacante tiene un servidor web de mentira (podría ser la IP del propio atacante si corre un Apache con página de prueba). Use el dominio que quiera probar (idealmente uno que la víctima vaya a consultar).

4. Inicie dnsspoof usando ese archivo: dnsspoof -i eth0 -f dnsfake.txt. Esto pone al atacante escuchando consultas DNS en la interfaz, y cuando vea una consulta para "www.ejemplo.com", responderá inmediatamente con la IP 192.168.1.100 en lugar de dejar que llegue la respuesta legítima. (Asegúrese de que *arpspoof* sigue corriendo para interceptar DNS queries de la víctima, o configure manualmente la víctima para usarlo).
5. En la máquina Víctima, limpie cualquier caché DNS local: en Windows ipconfig /flushdns, en Linux puede reiniciar *nscd* o simplemente asegúrese de que no ha resuelto ese nombre recientemente.
6. Ahora, en la víctima, intente acceder al dominio elegido: por ejemplo, abra un navegador y ponga http://www.ejemplo.com (asegúrese de usar *http* para no complicar con certificados) o simplemente haga un ping www.ejemplo.com.
7. Observe lo que ocurre: La víctima consultará DNS (likely su DNS real en internet). Pero el atacante, interceptando la consulta (via MitM), enviará una respuesta falsa antes/ en lugar de la verdadera. En la máquina víctima, el nombre www.ejemplo.com debería resolverse a 192.168.1.100 (la IP falsa del archivo). Si hace ping, verá que pingea a

192.168.1.100. Si abre el navegador, intentará cargar http://www.ejemplo.com pero en realidad irá a 192.168.1.100 (donde quizás el atacante muestre una página "falsa" de ejemplo).

8. Si el atacante tiene un servidor web en 192.168.1.100, podría mostrar un mensaje para confirmar que la víctima llegó allí (por ejemplo, una simple página HTML "Has sido DNS spoofeado"). Si no tiene servidor, el navegador de la víctima quizá muestre error de conexión (porque intentó a 192.168.1.100 y no obtuvo respuesta HTTP). En ambos casos, notamos que no fue al sitio real.
9. En la terminal de dnsspoof, probablemente verá log de la captura/respuesta: líneas indicando que interceptó una consulta A para "www.ejemplo.com" y envió la IP definida.
10. Detenga dnsspoof (Ctrl+C) y arpspoof cuando termine.

Comandos necesarios:
En Atacante:

Crear archivo de hosts falsos (contenido: "IP_falsa nombre"):

echo "192.168.1.100 www.ejemplo.com" > dnsfake.txt

dnsspoof -i eth0 -f dnsfake.txt # Lanzar suplantador DNS en interfaz eth0 usando ese archivo

En Víctima:

ipconfig /flushdns # (Windows) Limpiar caché DNS

ping www.ejemplo.com # Consultar el dominio (o abrir en navegador)

Riesgos asociados y mitigación:

- **Riesgo:** La **suplantación de DNS** permite ataques de phishing/redirección muy peligrosos: la víctima cree estar accediendo a un sitio legítimo pero en realidad llega al servidor del atacante. Esto combinado con un sitio falso convincente puede robar credenciales u otra info.
  Mitigación: Implementar **DNSSEC** en las resoluciones (firmas digitales que garantizan la autenticidad de la respuesta DNS), de modo que una respuesta falsificada sea detectable (el cliente la rechazaría al no poder validarla). Usar también **DoT/DoH** (DNS sobre TLS/HTTPS) dificulta que un atacante local intercepte consultas DNS fácilmente, pues van cifradas. Además, configurar equipos para usar siempre conexiones cifradas (HTTPS) ayuda: aunque la víctima sea llevada al servidor del atacante, el certificado HTTPS no coincidirá y saltará alerta en el navegador, avisando al usuario. Finalmente, monitorear en la red (un IDS podría detectar que se están enviando respuestas DNS no provenientes del servidor DNS verdadero, o múltiples ARP spoofs como preludio).

Resultados esperados:

- En la máquina Víctima, la resolución de www.ejemplo.com debe apuntar a la IP falsa que definimos (192.168.1.100). Esto se verifica porque, por ejemplo, el comando ping muestra Haciendo ping a www.ejemplo.com [192.168.1.100] en Windows (o en Linux PING www.ejemplo.com (192.168.1.100) ...). Es decir, el nombre se resolvió incorrectamente al IP del atacante.
- Si se probó en navegador y el atacante tenía un servidor web escuchando en 192.168.1.100, la víctima verá una página distinta a la real (o un mensaje de suplantación). Si no había servidor, la víctima verá error de conexión (lo cual ya es indicador de que algo anda mal si esperaba el sitio real).
- En la consola de dnsspoof, se verá algo como:
  dnsspoof: [DNS] *victim_ip*:*port* -> *dns_server_ip*:53 UDP (www.ejemplo.com)
  dnsspoof: spoofed reply sent for "www.ejemplo.com" to *victim_ip*
  Confirmando que se envió la respuesta falsa.
- La víctima en general no recibe indicación explícita de que fue engañada. A menos que note la IP (poco probable para un usuario común) o en caso de HTTPS, vería un *certificado no válido*. Pero con HTTP solito, no hay alerta.

Solución guiada:
Mediante *dnsspoof*, el alumno logró que la Víctima resolviera un nombre de dominio a la IP que el Atacante quiso. En esencia, el Atacante se hizo pasar por servidor DNS para esa petición. Esto demuestra lo fácil que es redirigir tráfico cuando se controla la resolución de nombres. Un usuario escribe "banco.com", y sin saberlo puede estar yendo al servidor del atacante.

En el laboratorio, quizás usamos un dominio ficticio; en la realidad, un atacante puede apelar a nombres comunes (por ejemplo, suplantar *"mail.empresa.com"* para interceptar correo web corporativo). El riesgo es enorme: combinando esto con un sitio falso, los usuarios entregarían sus contraseñas corporativas creyendo estar en el portal real.

DNSSEC se diseñó para combatir esto: los registros DNS vienen firmados digitalmente por la autoridad del dominio. Si la víctima tiene validación DNSSEC habilitada (y su resolver lo soporta), entonces la respuesta no autenticada del atacante sería ignorada (porque no viene con firma o con firma incorrecta). Sin embargo, DNSSEC aún no es universal en todos los servicios y clientes.

Otra mitigación mencionada es **DoT/DoH** (DNS sobre TLS/HTTPS). Si la víctima resuelve usando, digamos, DNS sobre HTTPS a un servidor público (como 1.1.1.1 con DoH), el atacante en la LAN no podría leer ni modificar esas consultas fácilmente ya que van cifradas a través de TLS. Eso impediría este ataque directo, aunque si el atacante está en MitM,

podría intentar bloquear esas conexiones DNS seguras (ataque de denegación) pero no falsificarlas tan fácilmente.

Además, se sugiere la capa de aplicación segura: en un caso real, suponga que redirigimos *"www.google.com"* a un servidor falso. Si el usuario intenta **HTTPS**, el navegador del usuario recibiría un certificado distinto (no el verdadero de Google), y mostrará una alerta de seguridad (*"la identidad del sitio no se puede verificar"*). Un usuario entrenado sabría que eso indica un ataque. Por tanto, **el uso de HTTPS en todos los sitios críticos reduce el efecto** de DNS spoofing, porque aunque lo redirijan, no podrán imitar fácilmente un certificado válido. No obstante, usuarios poco atentos podrían ignorar la alerta y proceder (pésima idea), pero ahí entra la concienciación.

El alumno pudo comprobar la redirección observando que el ping del dominio iba a 192.168.1.100 en lugar de su IP real. En una auditoría, hacer este tipo de prueba con permiso puede evaluar la robustez de los servicios DNS de la empresa (por ejemplo, si tienen DNSSEC). Sin permiso, obviamente sería malicioso.

Cabe destacar que este ataque fue posible porque previamente estabamos en MitM interceptando las peticiones DNS (o posicionado como DNS). Esto resalta la cadena de explotación: primero ARP Spoof -> luego DNS Spoof -> luego Phishing/hijack. Muchas brechas se logran encadenando así vectores.

El alumno, tras este ejercicio, entenderá la importancia de **la integridad de la resolución de nombres**. DNS es un pilar de Internet: si se compromete, uno puede dirigir a la gente a lugares incorrectos sin su conocimiento. Por eso existen esfuerzos como DNSSEC, y también por eso los navegadores exigen certificados válidos (como segunda capa de verificación).

En resumen, con *dnsspoof* se demostró un ataque potente pero contrarrestable con buenas prácticas. En el informe de auditoría, se recomendaría: *Implementar DNSSEC en los dominios corporativos y usar resolvers validados; fomentar el uso de DNS cifrado; y asegurar que los usuarios solo confíen en sitios con certificados válidos*. Además, mantener segmentada la red para que un atacante interno no pueda envenenar DNS local (por ejemplo, usando switches que no dejen tráfico de la VLAN del atacante llegar al DNS server, etc.). Este es un ataque más de capa de red que exhibimos y su defensa se suma a la lista de capas a proteger en la organización.

Ejercicio 9: Monitoreo de transferencias de archivos con filesnarf (tráfico NFS)

Herramienta: filesnarf (suite dsniff)
Objetivo: Mostrar cómo es posible capturar información sobre **archivos** que se transfieren por la red en claro, utilizando la herramienta *filesnarf*. En particular, *filesnarf* se enfoca en sniffear archivos transmitidos via **NFS (Network File System)**, un protocolo de compartición de archivos en UNIX que originalmente no cifraba el tráfico. El objetivo es detectar nombres de archivos (e incluso contenido) que viajen sin cifrado, evidenciando fuga de metadatos y datos sensibles si no se protegen las comparticiones.

Instrucciones:

1. **Entorno NFS:** Prepare un escenario en que se use NFS sin cifrar. Por ejemplo, en una máquina "Servidor" Linux configure una exportación NFS de un directorio (editando /etc/exports, etc., y arrancando exportfs/nfs-kernel-server). En una máquina "Cliente" Linux, monte esa exportación con mount -t nfs <IP_servidor>:/ruta/exportada /mnt/compartido. Asegúrese de no usar opciones de cifrado (NFSv3 no cifra nada; NFSv4.2 puede cifrar pero supondremos no).
2. En la máquina Atacante (que debe estar en la misma red; si replicamos mitM, aplicar ARP spoof entre Cliente y Servidor para capturar su tráfico NFS), ejecute filesnarf -i eth0 -v. El parámetro -v (verbose) hace que liste en la salida los nombres de los archivos que va interceptando. Sin -v, *filesnarf* podría intentar guardar copias de los archivos directamente en el directorio actual, lo cual también puede suceder.
3. Ahora, en la máquina Cliente, realice alguna acción en el recurso NFS compartido para generar tráfico:
    - Por ejemplo, navegue al directorio montado (/mnt/compartido) y ejecute ls -l para listar archivos, o abra/copíe un archivo del share (por ejemplo, cat /mnt/compartido/archivo.txt o cp /mnt/compartido/archivo.txt /tmp/).
    - Esto hará que por la red pasen las peticiones de lectura/escritura NFS y los nombres de los archivos accesados.
4. Observe la salida de filesnarf: debería listar el/los nombres de archivos a los que se accedió. Por ejemplo:
5. filesnarf: 192.168.1.60:/export/docs/report.pdf -> saved as 192.168.1.60_export_docs_report.pdf

o en modo verbose podría mostrar algo como:

/export/docs/report.pdf (NFS file open)

/export/docs/report.pdf (NFS file read 2048 bytes)

...

Esto indica que detectó la apertura/transferencia del fichero *report.pdf*. Incluso puede que lo guarde localmente.

6. Si se copió un archivo entero vía NFS, *filesnarf* posiblemente lo haya escrito en el directorio donde se ejecutó (con un nombre basado en la IP y ruta como en el ejemplo arriba). Verifique si existe el archivo y compárelo con el original: en muchos casos, filesnarf habrá capturado el contenido completo (salvo que la transferencia sea muy grande o haya limitaciones).
7. Pruebe con otro archivo o acción (crear un archivo nuevo, etc.) para ver cómo lo refleja la herramienta.
8. Detenga filesnarf después de la prueba.

Comandos necesarios:
En Atacante:

```
filesnarf -i eth0 -v            # Escuchar en interfaz eth0 tráfico NFS verbosamente
```

(La configuración de NFS servidor/cliente con exportfs y mount no la detallamos por ser más compleja, se asume conocida para el contexto del ejercicio. Lo importante es generar tráfico NFS.)

Riesgos asociados y mitigación:

- **Riesgo:** Las comparticiones de archivos sin cifrar (ej: NFSv3, FTP, SMBv1 sin cifrado) permiten que un atacante vea qué archivos se transfieren, sus nombres e incluso su contenido. Esto es una grave filtración de **metadatos** (nombres de proyectos, usuarios, etc.) y potencialmente de datos sensibles (si captura el contenido). **Mitigación:** Utilizar **protocolos seguros de compartición**: NFS versión 4.2 puede utilizar cifrado de transporte; SMBv3 tiene cifrado de sesión; o encapsular el tráfico de compartición dentro de túneles seguros (VPN, SSHFS en lugar de NFS, etc.). Además, segmentar la red de almacenamiento (por ejemplo, VLAN separada para NFS a la que clientes normales no acceden) reduce que un atacante promedio pueda sniffar ese tráfico. En general, cualquier servicio de transferencia de ficheros debería cifrarse (SFTP, HTTPS, etc. en su defecto).

Resultados esperados:

- *filesnarf* mostrará los **nombres de archivos** a los que se accede via NFS. Por ejemplo, si el cliente listó y abrió "secreto.txt", la herramienta podría al menos indicar "/ruta/secreto.txt". Ese es un metadato importante: el atacante ahora sabe que existe un archivo llamado "secreto.txt". Si lo guardó, tendrá su contenido.
- En caso de guardar archivos, el atacante podría acabar con una copia del mismo en su máquina sin haber tocado el servidor directamente. Esto sucede porque NFS transmite los datos de archivo en claro (RPC READ/WRITE operations) y *filesnarf* los reconstruye.

- Si no llegó a capturar todo el contenido (posible si no se hizo lectura completa, etc.), igual habrá evidencias de nombres y tamaños.
- Por ejemplo, output podría ser:
- filesnarf: 192.168.1.60:/shared/secret.doc -> saved as 192.168.1.60_shared_secret.doc
- filesnarf: 192.168.1.60:/shared/photos.zip -> saved as 192.168.1.60_shared_photos.zip

Y estos archivos estar en el directorio actual.

- A nivel usuario, el cliente NFS no notó nada, todo funcionó normal. Solo que el atacante silenciosamente copió en espejo lo que pasaba.

Solución guiada:
Este ejercicio expuso que **los protocolos de compartición sin cifrar comprometen tanto la confidencialidad como la privacidad de la información**. El alumno vio que simplemente escuchando tráfico NFS pudo obtener una **lista de archivos** que otro equipo estaba leyendo o escribiendo. Por ejemplo, suponga que en la salida vio "HR-Salaries.xlsx" – ya ese nombre revela información sensible (Departamento de RRHH, salarios). Y si el contenido viajó completo, el atacante podría tener los datos salariales íntegros.

En la práctica, *filesnarf* demostró cómo un atacante puede llevar a cabo un espionaje pasivo de actividades de archivo. En entornos reales, esto equivaldría a alguien en la misma red corporativa pudiendo interceptar transferencias de documentos confidenciales entre un servidor de archivos y los clientes. Claramente es una situación inaceptable.

Las mitigaciones son claras: modernizar protocolos o protegerlos. NFSv4 implementa mejoras, entre ellas la posibilidad de seguridad a nivel de transporte (en combinación con Kerberos y cifrado). SMB a partir de v3 (Windows 2012+) tiene opción de cifrado de extremo a extremo en shares. Si por legado se sigue usando NFSv3, una medida es encapsularlo en una VPN interna (por ejemplo, IPsec entre cliente y servidor, de modo que aunque un atacante sniff, todo va cifrado a nivel IP). **Usar SFTP/SSHFS** es otra alternativa para pequeñas escalas en vez de NFS.

Además, *filesnarf* específicamente es para NFS, pero otros protocolos de archivos también son vulnerables: por ejemplo, **FTP** (no solo credenciales como vimos, sino archivos si se transfieren, un sniffer puede reconstruirlos también) y **HTTP** descuidado (descargas via HTTP pueden ser interceptadas y reconstruidas). Wireshark de hecho tiene capacidad de "Follow TCP stream" y extraer archivos de ciertos protocolos (HTTP, SMB), similar a lo que filesnarf hace.

Entonces, la lección es amplia: **los datos en tránsito deben cifrarse**, no solo las contraseñas. Aquí vimos que aunque la autenticación quizás ni existe (NFS suele confiar en IP y permisos, no manda contraseñas en la operación), el contenido estaba expuesto.

El alumno, al comprobar que filesnarf quizás guardó una copia del archivo interceptado, entenderá que un atacante ni siquiera necesita robar credenciales o acceder al servidor: puede literalmente **robar la información en el aire**. Este es un concepto poderoso sobre la vulnerabilidad de la información en tránsito.

Se refuerza así la importancia de estrategias de seguridad como **cifrado de datos en movimiento**, **segmentación de redes sensibles** (por ejemplo, poner servidores de ficheros en subredes aisladas accesibles solo por ciertos clientes, dificultando la presencia de un sniffer no autorizado) y **auditoría de tráfico** (detectar si hay un volumen anormal de sniffing o respuestas ARP raras).

Este ejercicio también complementa los anteriores: habíamos visto robar credenciales (dsniff), ahora robar archivos directamente. Un atacante real probablemente haría todo junto: primero se pone en MitM, luego captura contraseñas (dsniff) para acceder a algunos servicios, y simultáneamente captura archivos en protocolos no cifrados. Esto demuestra la cadena de peligros una vez que la red interna es comprometida.

En informes de auditoría, tras identificar NFS sin protección, recomendaríamos: *"Implementar NFSv4 con krb5i/krb5p (integridad y privacidad) para cifrar el tráfico de archivos, o migrar a soluciones de transferencia seguras; Considerar segmentar la red de servidores de ficheros y habilitar TLS en todas las comunicaciones de intercambio de archivos."*.

sudo filesnarf -i eth0 -v

```
~ ❯ sudo filesnarf -i eth0 -v
filesnarf: listening on eth0
```

```
229 Entering Extended Passive Mode (|||1032|)
550 Access denied.
ftp> get readme.txt
local: readme.txt remote: readme.txt
229 Entering Extended Passive Mode (|||1033|)
125 Data connection already open; starting 'BINARY' transfe
r.
100% |**************|   379        7.85 MiB/s    00:00 ETA
226 Transfer complete.
379 bytes received in 00:00 (7.08 KiB/s)
ftp>
```

Ejercicio 10: Ataque de flooding de tabla CAM con macof (MAC Flooding)

Herramienta: macof (suite dsniff)
Objetivo: Simular un ataque de **MAC Flooding** en un switch usando *macof*, que envía una gran cantidad de tramas con direcciones MAC de origen aleatorias para llenar la tabla CAM del switch. El objetivo es ver qué ocurre cuando el switch no puede aprender más direcciones: típicamente empieza a comportarse como un **hub**, enviando el tráfico a todos los puertos, lo que podría permitir sniffing generalizado (además de causar degradación de la red por la inundación). Este ejercicio ayuda a comprender una vulnerabilidad a nivel 2 (conmutación) y la importancia de las contramedidas en switches.

Instrucciones:

1. Identifique un segmento de red conmutado para la prueba. Idealmente use un switch no crítico o un entorno virtual que simule switching. *macof* generará miles de direcciones MAC falsas, lo que puede sobrecargar algunos dispositivos; en un lab físico, téngalo en cuenta.
2. En la máquina Atacante, simplemente ejecute macof (por defecto envía por eth0; si no, especifique -i eth0). Este comando empezará a arrojar decenas de miles de tramas por segundo, cada una con un origen MAC distinto y un destino aleatorio en la subred (usualmente IP incrementales). Verá en la consola desfilar líneas de direcciones generadas, o quizás no muestre nada pero estará enviando.
3. Deje macof corriendo durante unos 30 segundos a 1 minuto. Durante este tiempo:
    - Observe el comportamiento de la red: si hay otros equipos en el switch, podrían experimentar lentitud o perturbaciones. Por ejemplo, intente hacer ping entre dos máquinas cualesquiera en ese switch: es posible que aumente la latencia o que de pronto lleguen pings de fuentes inesperadas.
    - Si tiene acceso al switch (interfaz de gestión), compruebe su tabla de direcciones MAC (show mac address-table en Cisco por ej.). Verá que está llena de entradas (posiblemente todas apuntando al puerto del atacante, ya que envió con origenes falsos por ahí).
    - En algunos switches, una vez saturada la CAM, comenzarán a difundir todo broadcast y tráfico desconocido por todos los puertos. Para probar esto, en otra máquina de la red intente generar tráfico unidireccional (por ejemplo, envíe un archivo a otra máquina via netcat o ftp) mientras macof está activo; luego en la máquina atacante abra Wireshark a ver si captura paquetes de esa transferencia que normalmente no debería ver. Si el switch "falló abierto", el atacante verá tráfico ajeno.

4. Tras un minuto, detenga macof (Ctrl+C). Es probable que la red se normalice después de unos instantes (los switches limpian la tabla y vuelven a aprender). Puede que algunos switches incluso se bloqueen temporalmente o puertos entren en modo seguridad si lo tienen.
5. Registre lo observado: ¿Hubo pérdida de paquetes? ¿El atacante logró ver tráfico de terceros (indicando broadcast masivo)? ¿Se saturó CPU del switch (a veces estos ataques consumen CPU de switch)?
6. *Nota:* En entornos virtuales (como VirtualBox/VMware vswitch), puede que la emulación de switch no sea perfecta y macof no provoque un "hub" porque ya es más hub-like. En hardware real con un switch no protegido, sí sucede.

Comandos necesarios:
En Atacante:

```
macof -i eth0          # Generar tráfico MAC flooding en interfaz eth0
```

Riesgos asociados y mitigación:

- **Riesgo:** El MAC flooding es un ataque de **Denegación/Degradación de Servicio** a nivel red, y además un facilitador de sniffing. Al sobrecargar la tabla CAM, un atacante puede conseguir que tráfico que debía ser conmutado (unicast) sea enviado a todos los puertos, **permitiéndole capturarlo**. También puede saturar el switch y causar lentitud o caída de red (porque manejando miles de MACs por segundo, algunos switches agotan memoria o CPU).
  Mitigación: Configurar **port security** en los switches: limitar el número de direcciones MAC permitidas por puerto (por ejemplo, no más de 2 MAC en un puerto de acceso). Esto haría que, cuando el atacante intente registrar cientos, el switch bloquee el puerto o ignore MACs extra. Muchos switches gestionados ofrecen esta característica. Otra mitigación es **monitorizar**: una ráfaga tan inusual de direcciones podría disparar alertas en sistemas de monitoreo de red. En redes bien segmentadas, un atacante tampoco debería tener fácil acceso físico a un puerto de switch para lanzar esto (pero un equipo comprometido internamente sí podría). En general, activar las funciones de seguridad de switch (como Cisco *Storm Control*, *Dynamic ARP Inspection*, etc., aunque DAI es para ARP, suelen acompañar port security).

Resultados esperados:

- Durante la ejecución de macof, probablemente la consola inundará (dependiendo de la versión) mostrando cada paquete generado: algo tipo macof: sent packet SRC=02:fa:... DST=ff:ff:... repetidamente. O puede no imprimir pero saturar la red.
- Si el ataque tuvo efecto: otros equipos en la red pueden notar anomalías. Por ejemplo:

    - La víctima de ejercicios anteriores, sin estar en MitM ahora, de repente recibe tráfico extraño (puede que su Wireshark note paquetes ARP o demás de direcciones raras).
    - El atacante podría abrir Wireshark después de unos segundos y ver tráfico que no le corresponde (por ejemplo, ARPs entre otras máquinas, o respuestas de dispositivos a solicitudes de la víctima, etc., indicio de que el switch está inundando).
    - En el peor caso, la conectividad se vuelve errática: pings con fluctuaciones, etc.
- Si el switch tenía port security, es posible que tras unas cuantas, bloquee el puerto del atacante (lo *shutdown*). En tal caso, la máquina atacante perderá conectividad (no verá nada, incluso macof deja de tener efecto porque el puerto se cerró). Eso es buena señal de mitigación.
- Tras parar macof, la red debería volver a la normalidad rápidamente (si no hay puertos err-disabled, etc.).
- Este ataque no produce "prueba" tan visible al usuario final salvo la degradación; su efectividad se mide en si permitió sniffing. Lo ideal es que el alumno haya logrado capturar algo que antes no podía (por ejemplo, un ping entre dos terceros visible). Si lo logró, confirmamos que el switch actuó como hub.

Solución guiada:
El ejercicio con *macof* revela cómo un atacante puede explotar un switch mal configurado para **anular su principal ventaja de seguridad** (la segregación de tráfico). El alumno habrá notado que, tras inundar la red con direcciones MAC falsas, el switch comenzó a comportarse de forma anómala. En una situación ideal de demostración, el atacante habría podido ver comunicaciones de otros equipos que antes no estaban accesibles, lo que significa que logró romper el aislamiento. También es probable que la red se ralentizara o se perdieran paquetes durante la inundación, manifestando el aspecto de DoS.

La mitigación, *port security*, se entiende fácil tras ver el ataque: si un puerto solo acepta, digamos, 5 MACs distintas, en cuanto macof intente la 6ª, el switch sabrá que es un comportamiento inusual y podrá bloquearlo. Muchos switches incluso mandarían una alarma SNMP al admin de que se detectó un "MAC limit exceeded", alertando de un posible ataque.

Otra reflexión es que muchos switches modernos también implementan contramedidas específicas: por ejemplo, *MAC address throttling* o *IEEE 802.1X* para autenticar dispositivos antes de darles acceso a la red. Con 802.1X, un dispositivo no autenticado no podrá enviar tráfico como macof. Sin esas medidas, un simple host dentro de la red puede ejecutar este ataque con herramientas disponibles (dsniff en este caso).

El alumno también pudo ver que este ataque no roba datos per se, pero habilita el sniffing general. Es un medio para un fin: hacerlo, luego ejecutar Wireshark/dsniff y capturar cosas (como credenciales en tráfico que de otra forma no habrían pasado por el atacante). Este ataque era más común en redes antiguas con hubs o switches baratos. Hoy, con port security disponible incluso en switches de gama media, es menos frecuente, pero la amenaza persiste si no se configura.

Como conclusión, se refuerza: **los dispositivos de la capa 2 deben configurarse con medidas de seguridad**. Port security es mencionada en la política del ejercicio, y es de las primeras. Adicionalmente, se puede combinar con *VLANs privadas* o *seguras* para aislar puertos entre sí. El estudiante comprenderá que la seguridad no es solo a nivel software; aquí vimos un ataque puramente de infraestructura.

En la auditoría de un entorno, un pentester podría intentar un MAC flooding para ver si logra sniffar tráfico. Si lo logra, reportaría la falta de port security como hallazgo. Si no (porque el puerto se bloqueó), felicitaría esa configuración. Por tanto este ejercicio cubre esa área.

También evidenció un concepto: la **negación de servicio**. Aunque el atacante no estuviera interesado en sniffing, macof por sí puede ser usado para simplemente derribar o saturar la red local (impactando la disponibilidad). Los sistemas de detección de intrusos podrían ver esta tormenta de tramas y reaccionar. Por ende, es un patrón de ataque detectable si se monitoriza volumen en la red.

En resumen, con *macof* el alumno vio el delicado equilibrio en switches entre rendimiento y seguridad, y cómo **ataques de nivel 2** pueden tener consecuencias serias (exfiltración de datos o interrupción de red). La mitigación recomendada es implementar **controles en los switches** (limitar MACs por puerto, etc.) para evitar la sobrecarga deliberada.

sudo macof -i eth0

```
~ ❯ sudo macof -i eth0
ae:d2:df:1:5a:b1 e6:a4:2c:11:8f:c5 0.0.0.0.23757 > 0.0.0.0.22149: S 1421381427:1421381427(0) win 512
a3:73:13:6c:73:dc 66:87:19:5d:5:ab 0.0.0.0.38111 > 0.0.0.0.11112: S 893050272:893050272(0) win 512
ea:2c:ca:e:df:65 aa:9a:25:76:63:58 0.0.0.0.18016 > 0.0.0.0.53046: S 239837017:239837017(0) win 512
cb:b0:ad:58:7:72 79:b7:58:15:b1:55 0.0.0.0.15266 > 0.0.0.0.37987: S 219281443:219281443(0) win 512
a6:89:d4:1a:c6:6b 49:a7:4f:3b:c:11 0.0.0.0.57293 > 0.0.0.0.43946: S 1351742227:1351742227(0) win 512
d4:45:59:52:fe:1f 4c:23:ef:5:7e:e0 0.0.0.0.14648 > 0.0.0.0.48833: S 374648398:374648398(0) win 512
df:c4:40:1e:19:5f 98:4a:1c:7c:58:7c 0.0.0.0.7670 > 0.0.0.0.61549: S 441194129:441194129(0) win 512
43:bb:51:66:37:a0 df:81:5f:7f:8d:62 0.0.0.0.10808 > 0.0.0.0.37136: S 1044071509:1044071509(0) win 512
7d:4e:55:46:29:8a 49:3d:79:7d:c9:71 0.0.0.0.20705 > 0.0.0.0.41952: S 469660779:469660779(0) win 512
71:7c:a9:22:4b:39 49:5e:85:42:9:c7 0.0.0.0.18743 > 0.0.0.0.54170: S 1352467330:1352467330(0) win 512
b3:e9:3a:6d:c5:be 0:0:45:d:93:6b 0.0.0.0.1086 > 0.0.0.0.19338: S 1500004948:1500004948(0) win 512
d6:68:38:44:d1:90 bd:d2:1a:14:1f:a6 0.0.0.0.23498 > 0.0.0.0.21372: S 602333509:602333509(0) win 512
ac:52:b3:7c:b6:5d 80:9:da:50:f6:b0 0.0.0.0.50186 > 0.0.0.0.28150: S 632525502:632525502(0) win 512
be:cd:52:7b:71:7c 52:86:26:62:be:cd 0.0.0.0.32514 > 0.0.0.0.60482: S 1624372015:1624372015(0) win 512
d6:34:5a:48:5:5c 27:52:11:3c:93:7 0.0.0.0.17656 > 0.0.0.0.24046: S 1363388479:1363388479(0) win 512
24:2e:f3:77:eb:fa f3:83:1a:34:a4:37 0.0.0.0.63176 > 0.0.0.0.28652: S 1017058984:1017058984(0) win 512
```

Ejercicio 11: Intercepción de correos electrónicos con mailsnarf

Herramienta: mailsnarf (suite dsniff)
Objetivo: Comprobar la posibilidad de capturar el contenido de correos electrónicos enviados o recibidos sin cifrado usando *mailsnarf*. Esta herramienta escucha el tráfico en protocolos de correo (SMTP, POP3, IMAP) en texto claro y extrae los mensajes completos. El objetivo es demostrar la **exposición de información sensible** (contenido de emails, adjuntos) cuando no se usa cifrado en el correo (ej. SMTP sin TLS, POP3 sin SSL).

Instrucciones:

1. Prepare el entorno de MitM como en ejercicios previos para poder sniffar el tráfico entre un cliente de correo y su servidor (por ejemplo, ARP spoofing entre una estación de trabajo y el servidor de correo de la organización, en el lab use un servidor de correo de prueba). O bien, configurar la máquina atacante como gateway/bridge de la víctima de correo.
2. En la máquina Atacante, ejecute mailsnarf -i eth0. Esto hará que espere en la interfaz designada y cada vez que detecte tráfico SMTP (25), POP3 (110) o IMAP (143) *no cifrado* con contenido de correo, lo irá imprimiendo o guardando. Por defecto, mailsnarf muestra cada email atrapado con cabeceras y cuerpo en texto.
3. Genere tráfico de email desde la víctima:
    - **Para SMTP (envío):** Configure en la víctima un cliente de correo (Outlook, Thunderbird) con un servidor SMTP *sin TLS* (por ej., puerto 25, sin STARTTLS) y envíe un correo de prueba (preferiblemente dentro del lab para no depender de internet). Por ejemplo, un correo "Hola, este es un email de prueba" enviado desde A a B.
    - **Para POP3/IMAP (recepción):** De igual manera, configure la cuenta en el cliente pero sin SSL en POP3 (110) o IMAP (143), y haga que recupere los correos (por ejemplo, pulsar "Send/Receive").
    - Alternativamente, se puede usar telnet manual: desde la víctima telnet mailserver 110 y hacer diálogo POP3: USER, PASS, RETR 1 para recuperar un mail. O en SMTP telnet mailserver 25 y escribir un correo con comandos SMTP (HELO, MAIL FROM, etc., aunque es más laborioso).
    - Lo importante es que haya un correo completo transitando.
4. Observe la salida de mailsnarf en el atacante. Debería capturar el texto del correo. Por ejemplo, para SMTP envíos, a partir del comando DATA hasta el . final, mailsnarf interceptará todo: debería ver las cabeceras (From, To, Subject) y el cuerpo del mensaje impreso en su terminal. Para POP3/IMAP recepciones, similar: mailsnarf presentará el contenido del email que pasó por la red hacia el cliente.

5. En el caso de adjuntos, mailsnarf no los guarda como archivos separados, pero vería el texto base64 posiblemente. Sin embargo, el ejercicio se enfoca en demostrar que *todo el correo*, incluso cuerpo y asunto, fueron visibles.
6. Una vez confirmada la captura (mailsnarf mostrará quizá el mensaje con una línea "*Caught POP3 email*" o "*SMTP*" etc.), detenga mailsnarf.
7. (Opcional) Si se quiere guardar los emails en archivos, mailsnarf tiene opciones para volcar cada mensaje a un archivo .mbox en disco en lugar de stdout. Pero para la demo, stdout es suficiente.

Comandos necesarios:
En Atacante:

mailsnarf -i eth0 # Escuchar en eth0 correos en protocolos sin cifrar

En Víctima (por ejemplo usando netcat/telnet para demostrar SMTP):

telnet 192.168.1.60 25

HELO victima.com

MAIL FROM:<alice@victima.com>

RCPT TO:<bob@victima.com>

DATA

From: alice@victima.com

To: bob@victima.com

Subject: Prueba

Hola Bob, este es un correo de prueba.

.

QUIT

(Lo anterior envía un correo muy básico. En la práctica, un cliente de correo haría esto automáticamente.)

Riesgos asociados y mitigación:

- **Riesgo:** Correos electrónicos enviados sin cifrado (SMTP sin STARTTLS) o recuperados sin cifrado (POP3/IMAP sin SSL) pueden ser **leídos íntegramente por un atacante** en la misma red. Esto expone contenido confidencial, anexos privados, etc., violando completamente la privacidad. Además, credenciales de correo también van en POP3/IMAP en claro (ya cubierto por dsniff antes).

Mitigación: Habilitar cifrado TLS en todos los servicios de correo. Usar SMTPS / SMTP STARTTLS para envío (hoy estándar), e **IMAPS/POP3S** para lectura. De este modo, aunque un atacante intercepte el tráfico, estará cifrado y no podrá leer los correos. También emplear protocolos modernos de autenticación (OAuth, etc.) en lugar de plaintext, pero lo principal es TLS. Adicionalmente, segmentar la red de servidores de correo y clientes o usar VPN para oficinas remotas evita tráfico de correo en texto plano por la red local. Pero realmente, hoy en día la **encriptación de capa transporte** es la solución esperada.

Resultados esperados:

- En la pantalla de mailsnarf, se debería ver el contenido de al menos un correo. Por ejemplo:
- 07/11/2025 12:34:56 SMTP 192.168.1.50 -> 192.168.1.60
- From: alice@victima.com
- To: bob@victima.com
- Subject: Prueba
- Date: Wed, 11 Jul 2025 12:34:50 +0200
- Hola Bob, este es un correo de prueba.

mailsnarf puede no mostrar cabeceras *Received* generadas, pero el principal cuerpo y campos principales estarán.

- Si se capturó via POP3, vería similar, quizás con la línea "Caught POP3 email from <server>" y luego el mismo contenido.
- En definitiva, **el atacante leyó el correo tal cual**. Si había información sensible (contraseñas, números, detalles internos), él lo tiene.
- El usuario víctima y los servidores no notan nada extraño, salvo que su correo no estaba protegido. Todo funciona normal. Solo que fue espiado.
- Si se envió varios correos, mailsnarf listaría uno tras otro.

Solución guiada:
El alumno habrá presenciado cómo un atacante en la red puede **leer comunicaciones de email sin necesidad de comprometer ninguna cuenta**, simplemente mediante sniffing avanzado. Esto subraya la importancia de la confidencialidad en el correo electrónico. Mucha gente piensa en proteger la cuenta (contraseña), pero aquí vimos que incluso con la contraseña segura, si la transmisión no va cifrada, el atacante puede leer el mensaje en tránsito.

Afortunadamente, a fecha de hoy la mayoría de clientes y servidores de correo soportan y suelen forzar TLS. Por ejemplo, Gmail, Outlook 365, etc., siempre usan SMTP STARTTLS y POP/IMAP sobre SSL. Sin embargo, en entornos corporativos o antiguos, a veces quedan servicios de correo internos sin cifrar (o cifrado mal configurado). La auditoría debe detectar si algún servidor permite comunicaciones no cifradas y corregirlo.

El riesgo no es solo teórico: mailsnarf nos mostró un correo; imagínese que era un correo de finanzas con números de tarjeta, o de legal con anexos confidenciales. Todo eso habría quedado expuesto. Además, la **integridad** también puede afectarse: un atacante en MitM podría incluso modificar el correo antes de reenviarlo (no con mailsnarf, pero con herramientas manuales) y ni emisor ni receptor lo sabrían fácilmente (a menos que firmen los correos con PGP, lo cual es poco común).

Mitigaciones: **SMTP STARTTLS** significa que el cliente y servidor negocian cifrado en el puerto 25 mismo; los admin deben asegurarse de que está habilitado y preferido. Algunos servidores incluso rechazan entrega si el receptor no soporta TLS (MTA-STS, etc.). Y a nivel cliente, configurar siempre IMAP/POP con SSL (993/995) y SMTP con TLS. De hecho, muchas guías de TI prohíben configuraciones inseguras.

También es importante la detección: un IDS podría detectar cadenas de texto típicas de correo (como "Subject:" en texto claro en el puerto 25), lo que indicaría que alguien envía correos sin cifrar. Eso debería generar alertas para administradores hoy en día.

El alumno con esto completa la visión de *sniffing*: no solo credenciales y archivos, sino también **comunicaciones enteras**. E-mail es solo un ejemplo; por analogía, cualquier protocolo de aplicación (HTTP, FTP, TELNET) sin cifrar sufre lo mismo. La regla dorada es: **cifrar todo**. Este ejercicio en particular suele causar impacto porque se ven mensajes personales/profesionales expuestos, algo que todos entienden.

En el reporte, se aconsejaría explícitamente: *"Configurar los servidores de correo para requerir TLS en todas las conexiones cliente-servidor y servidor-servidor. Deshabilitar los puertos/pop3/imap sin cifrado. Educar a los usuarios para no enviar información sensible por correo electrónico sin cifrado de extremo a extremo (PGP) a menos que confíen en la seguridad de la red, ya que aun con TLS un atacante con control de servidores podría leer, pero eso es otra capa).*

Así, el alumno ve cómo los diferentes hilos de sniffing convergen: ARP spoof -> mailsnarf => **compromiso total de la privacidad del correo en la red local**. La imagen global de los ejercicios 5,6,7,8,9,11 es que un intruso en la red local puede, sin romper contraseñas, recolectar gran cantidad de datos sensibles si no se emplean medidas de seguridad básicas de cifrado.

Ejercicio 12: Monitoreo de navegación web HTTP con urlsnarf

Herramienta: urlsnarf (suite dsniff)
Objetivo: Monitorear las **URLs de los sitios web** que los usuarios visitan a través de HTTP no cifrado utilizando *urlsnarf*. Esto permite a un atacante o auditor recopilar un historial de navegación (páginas exactas visitadas) de las víctimas en la red. El objetivo es remarcar el impacto en la privacidad: aun sin capturar todo el contenido, saber qué páginas consulta un usuario ya es información sensible (por ejemplo, sitios de salud, intereses, etc.), y mostrar la necesidad de usar **HTTPS** para proteger incluso las URLs.

Instrucciones:

1. Como en ejercicios anteriores, sitúe al Atacante en posición MitM (ARP spoof) para captar el tráfico HTTP de la Víctima hacia Internet, o configure su máquina como gateway.
2. En la máquina Atacante, ejecute urlsnarf -i eth0. Esto hará que la herramienta escuche todo tráfico HTTP (puerto 80) en la interfaz y extraiga las peticiones GET que contienen URLs. Por defecto, urlsnarf formatea la salida al estilo *Common Log Format* de Apache, incluyendo timestamp, host origen, URL solicitada, etc.
3. En la máquina Víctima, genere tráfico web HTTP. Por ejemplo:
    - Visite en el navegador varios sitios *que permitan HTTP plano*. Muchos redirigen a HTTPS hoy, así que podrías usar http://example.com, http://neverssl.com, tal vez algún sitio local que tengas en HTTP, o forzar manualmente http:// en otros sitios sabiendo que pueden responder.
    - También puede usar herramientas CLI: curl http://example.com desde la víctima, o lynx/w3m si están instalados, para visitar por HTTP.
    - Intente visitar al menos 2-3 URLs distintas (diferentes dominios o diferentes páginas de un mismo dominio) para ver cómo se registran.
4. Mire la salida de urlsnarf en el Atacante. Debería aparecer una línea por cada solicitud HTTP detectada. Por ejemplo:
5. 192.168.1.50 - - [19/Nov/2025:21:15:30 +0100] "GET http://example.com/index.html HTTP/1.1" 200 1256
6. 192.168.1.50 - - [19/Nov/2025:21:15:31 +0100] "GET http://example.com/logo.png HTTP/1.1" 200 5432
7. 192.168.1.50 - - [19/Nov/2025:21:16:10 +0100] "GET http://neverssl.com/online HTTP/1.1" 200 3000

Se ve la IP de la víctima, la fecha/hora, el método GET y la URL completa solicitada, así como código de estado y bytes (si pudo capturar la respuesta header). Básicamente, un registro de todas las páginas y recursos pedidos.

8. Si el usuario navegó varias páginas, se hará una lista. Este es esencialmente un *log de navegación* no autorizado.
9. Note que urlsnarf captura incluso los recursos (imágenes, scripts) pedidos via HTTP. Eso genera muchas líneas, pero la primera línea de cada dominio es la principal (HTML). Se podría filtrar luego por extension si se quisiera enfocarse solo en páginas.
10. Detenga urlsnarf cuando termine.

Comandos necesarios:
En Atacante:

```
urlsnarf -i eth0          # Escuchar peticiones HTTP y registrar URLs
```

(Navegación en víctima se hace manualmente en un navegador o con herramientas HTTP, no requiere comandos especiales aquí.)

Riesgos asociados y mitigación:

- **Riesgo:** Un atacante puede construir el **historial completo de navegación** de usuarios si éstos visitan sitios en HTTP. Las URLs revelan qué sitios y a veces qué contenidos específicos consulta un usuario (por ej., http://sitio.com/busqueda?consulta=salud+mental expondría términos privados). Esto invade la privacidad y puede facilitar ataques posteriores de ingeniería social (sabiendo intereses, etc.). **Mitigación: Adopción total de HTTPS** en la web. A estas alturas, cualquier sitio web debe ofrecer HTTPS y los navegadores ya marcan como "No seguro" el HTTP. Las empresas deben forzar navegación segura (por ejemplo, mediante proxies que bloqueen HTTP o re-escriban a HTTPS cuando sea posible). Además, se pueden usar herramientas como **HSTS (Strict Transport Security)** para que los navegadores no intenten ni siquiera HTTP. Para la parte del atacante interno, segmentar la red e implementar detección (un IDS podría notar gran cantidad de peticiones HTTP desde un mismo host sniffing) es secundario; lo principal es eliminar el tráfico en claro que da pie al problema.

Resultados esperados:

- El atacante obtiene líneas detalladas de cada URL visitada por la víctima. Esto incluye:
    - Los dominios (qué sitios web). Ej: example.com, neverssl.com, etc.
    - Las páginas exactas o recursos. Ej: /index.html, /images/logo.png, etc.

    - Cualquier parámetro en la URL. Ej: si la víctima buscó algo en un sitio sin HTTPS, la query aparecería. Por ejemplo: GET http://site.com/search?q=medical+condition expondría la consulta "medical condition".
    - Horarios de cada petición y tamaños de respuestas (no crítico para privacidad, pero contexto).
- Con esa información, el atacante puede deducir mucho: intereses personales, problemas de salud, qué servicios de banca se usan (si la víctima visitó http://oldbank.com/login sin HTTPS, sabría qué banco usa), etc. Incluso sin ver el contenido de la página, las URLs muchas veces describen bastante.
- Si repetimos: la víctima no nota nada. Solo el atacante registró silenciosamente todo.
- Si la víctima solo visita HTTPS (como ocurre en muchos sitios actuales), *urlsnarf no capturará nada útil*, porque no verá peticiones en puerto 80 con texto. Eso de hecho es la mitigación actuando.

Solución guiada:
Este ejercicio demostró la fuga de información que suponen las **conexiones HTTP no cifradas** incluso sin entrar a robar contraseñas o contenidos. Con *urlsnarf*, el alumno vio que es trivial generar un **registro detallado de la actividad web** de un usuario. Esto compromete la privacidad: por ejemplo, saber que alguien visita páginas de tratamientos médicos, o sitios de empleo, o redes sociales particulares, ya da mucha información que podría ser mal usada.

La mitigación es casi total hoy gracias a HTTPS. Si un usuario usa siempre HTTPS, *urlsnarf* se queda prácticamente ciego (podría ver dominio en SNI TLS a veces, pero no la URL completa con path, y en HTTP/2 ya ni eso si cifrado). Por eso, la comunidad ha empujado "HTTPS Everywhere". De hecho, herramientas como *urlsnarf* eran más útiles hace años; hoy en redes modernas uno ve muy poco tráfico HTTP puro. Aun así, siempre puede haber casos (dispositivos IoT comunicándose en claro, ciertas intranets viejas).

La respuesta defensiva complementaria es: las organizaciones deben **bloquear tráfico HTTP saliente** o redirigirlo. Muchos proxies corporativos fuerzan que toda petición se haga por ellos; si ven una a http://, podrían o bien bloquearla o reescribirla a https:// si saben que existe. Esto disminuye los escenarios explotables.

De los ejercicios, este es quizás el menos "crítico" en términos de seguridad inmediata (no es robar password ni datos sensibles), pero sí es un gran problema de **privacidad** y puede ser preludio de ataques. Por ejemplo, un atacante sabiendo que la víctima visita un cierto sitio web interno (porque ve GET http://intranet/dept/finance/report), podría intentar atacar ese sitio específico o engañar a la víctima con phishing referido a ese contexto.

Para el alumno, queda claro que **nada en HTTP es secreto**: ni siquiera las URLs. Y que con suficientes datos de URL, se puede perfilar a una persona u organización. Esto refuerza la idea de cifrar también las peticiones DNS (que revelan dominios) y usar HTTPS para ocultar paths.

Como notamos: en cada sniffing, la encriptación es la respuesta repetida. Este no es la excepción: la política "forzar HTTPS" es recomendada. De hecho, muchos navegadores modernos implementan ya DNS sobre HTTPS y HSTS preloaded lists para ciertos dominios, todo con tal de impedir que ataques de este estilo sean factibles.

Con la realización de estos 12 ejercicios centrados en sniffing/MITM, el alumno habrá obtenido una visión completa de por qué las mejores prácticas de seguridad en red existen y cómo las fallas se aprovechan con herramientas que están disponibles para cualquiera (dsniff suite, etc.). Cada ejercicio mostró un vector y se reforzaron las mitigaciones correspondientes citando las fuentes (por ejemplo, TLS, DNSSEC, port security).

*(Continuaríamos con ejercicios de escalada en complejidad, ahora en áreas de **scanning de vulnerabilidades (OpenVAS, Nexpose)**, **ataques activos (fuerza bruta con Brutus/John)**, **cracking (RainbowCrack)**, etc., aplicando formato similar.)*

sudo urlsnarf

```
~ ❯ sudo urlsnarf
[sudo] password for kali:
urlsnarf: listening on eth0 [tcp port 80 or port 8080 or port 3128]
192.168.0.10 - - [20/Nov/2025:12:17:22 +0100] "GET http://google.com/ HTTP/1.1" - - "-" "Mozilla/5.0 (X11; Li
4; rv:140.0) Gecko/20100101 Firefox/140.0"
192.168.0.10 - - [20/Nov/2025:12:18:14 +0100] "GET http://neverssl.com/ HTTP/1.1" - - "-" "Mozilla/5.0 (X11;
_64; rv:140.0) Gecko/20100101 Firefox/140.0"
192.168.0.10 - - [20/Nov/2025:12:18:14 +0100] "GET http://lushbrightsilvermorning.neverssl.com/ HTTP/1.1" - -
illa/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
192.168.0.10 - - [20/Nov/2025:12:18:14 +0100] "GET http://lushbrightsilvermorning.neverssl.com/online/ HTTP/1
-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
192.168.0.10 - - [20/Nov/2025:12:18:15 +0100] "GET http://lushbrightsilvermorning.neverssl.com/favicon.ico HT
 - "http://lushbrightsilvermorning.neverssl.com/online/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/201
efox/140.0"
```